



Facultad de Ingeniería
Escuela de Ingeniería Informática

BIOINSPIRADOS CON 'CONCIENCIA': UN ENFOQUE DE EXPLICABILIDAD PARA ANTICIPAR EL ESTANCAMIENTO

Por

Maximiliano Aguirre Faúndes

Trabajo realizado para optar al Título de
INGENIERO (CIVIL) EN INFORMÁTICA

Prof. Guía: Rodrigo Olivares Órdenes

Prof. Co-Referente: Pablo Olvares

Mayo 2026

Resumen

Los algoritmos de inteligencia de enjambres, inspirados en fenómenos o comportamientos biológicos, han demostrado ser de gran utilidad en la resolución de problemas de optimización. No obstante, la mayoría de estos algoritmos presentan una limitación significativa: la tendencia a la convergencia temprana y al estancamiento en óptimos locales. Para mitigar este problema, se recurre comúnmente al ajuste manual de parámetros basado en el criterio de experto, una práctica que ha demostrado ser ineficaz, ya que a menudo rompe el delicado equilibrio entre la exploración y la explotación que caracteriza a estos algoritmos. El presente trabajo de tesis propone integrar técnicas de explicabilidad para desvelar el funcionamiento interno de estos algoritmos, transformándolos de una 'caja negra' en un sistema transparente. El objetivo es desarrollar un controlador en línea capaz de justificar sus decisiones, lo que permitirá evitar los estancamientos. Para validar la implementación y el funcionamiento del controlador propuesto, se aplicará el algoritmo modificado a escenarios de logística de última milla, validados previamente por bibliografía.

Índice general

Resumen	II
1. Introducción	1
2. Marco Conceptual y Estado del Arte	3
2.1. Marco Conceptual	3
2.1.1. Heurísticas	3
2.1.2. Metaheurísticas	3
2.1.3. Optimización por enjambre de partículas	4
2.1.4. Algoritmos bio-inspirados	5
2.1.5. SeaHorse Optimization Algorithm	5
2.1.6. Exploración - explotación	5
2.1.7. Parámetros de ajuste en algoritmos bio-inspirados	6
2.1.8. Convergencia prematura y estancamiento	7
2.1.9. Hiperheurísticas	7
2.1.10. Explicabilidad	7
2.1.11. XAI (Inteligencia Artificial Explicable)	8
2.1.12. LIME - Local Interpretable Model-agnostic Explanations	8
2.1.13. Explicabilidad post ejecución	8
2.1.14. Métricas de diagnóstico	8
2.1.15. Programación lineal	9
2.1.16. Problemas de optimización computacional	9
2.1.17. Variables discretas y continuas	10
2.1.18. Problemas NP-hard	10
2.1.19. Congress on Evolutionary Computation - CEC	10
2.1.20. Criterios estadísticos para la evaluación de metaheurísticas	11
2.1.21. Controlador en línea	11
2.2. Estado del Arte	11
2.2.1. Metodología, criterios de inclusión y exclusión	12

2.2.2.	Aplicaciones de metaheurísticas en la resolución de problemas actuales	13
2.2.3.	Trabajos más recientes con SeaHorse Optimization Algorithm . . .	15
2.2.4.	Mejoras realizadas a las metaheurísticas	17
2.2.5.	Mejoras realizadas a las metaheurísticas, con explicabilidad	20
2.2.6.	SeaHorse Optimization Algorithm y LIME	23
3.	Definición del Problema	24
3.1.	Formulación del Problema	24
3.2.	Solución Propuesta	24
3.3.	Objetivos	25
3.3.1.	Objetivo General	25
3.3.2.	Objetivos Específicos	25
3.4.	Metodología	25
4.	Diseño de la solución	28
4.1.	Solución del problema	28
4.1.1.	Arquitectura y funcionamiento del controlador	28
4.1.2.	Métricas a monitorear	30
4.1.3.	Módulo XAI	31
4.1.4.	Ciclo de control	31
4.1.5.	Validación del controlador	32
4.2.	Técnicas de análisis	32
5.	Experimentación	34
5.1.	Diseño de experimentos	34
5.1.1.	Diseño para la etapa de experimentación	34
5.1.2.	Definición de Variables y Base para la Experimentación	35
5.2.	Casos de Estudio y Benchmarks	36
5.2.1.	Funciones de Benchmark Estándar (CEC)	36
5.2.2.	Caso de Estudio Aplicado (Optimización de Última Milla)	37
5.2.3.	Protocolo de Validación e Hipótesis	37
5.3.	Interpretación de Resultados	38
5.3.1.	Técnicas de Visualización de Rendimiento	38
5.3.2.	Interpretación y Análisis Estadístico	39
	Bibliografía	40

Índice de figuras

3.1. Metodología ilustrada	27
4.1. Funcionamiento base de una metaheurística.	29
4.2. Arquitectura general del controlador en línea propuesto.	30
5.1. Diseño experimental a alto nivel para la evaluación del algoritmo. Cada instancia de problema (Benchmark/Problema de Optimización) se ejecuta N veces para análisis estadístico.	35

Índice de tablas

2.1. Resumen de trabajos encontrados.	15
2.2. Resumen de trabajos aplicando SHO	17
2.3. Resumen de trabajos hiperheurísticos	20
2.4. Resumen metaheurísticas y explicabilidad.	23

Capítulo 1

Introducción

Los algoritmos de optimización computacional se encuentran frecuentemente descritos en una amplia variedad de ámbitos y ciencias, como por ejemplo, en la resolución de problemas de industria emergente [1]. Estos desafíos se caracterizan por su naturaleza compleja, siendo a menudo clasificados como problemas del tipo **NP-Hard**. Debido a su alto costo computacional, que se debe a que las soluciones potenciales aumentan exponencialmente, se vuelven inviables de resolver de forma exacta y con técnicas de modelamiento matemático tradicional [2]. Un acercamiento más moderno para darle solución a estos problemas es la resolución mediante algoritmos **bio-inspirados** los cuales tienen su raíz en la optimización por enjambre de partículas (PSO), basándose en el comportamiento colectivo (por ejemplo, el algoritmo usado en este trabajo, el cual es SeaHorse optimization algorithm [3]). Finalmente, los algoritmos bio-inspirados toman ciertas decisiones y generan confianza en sus resultados; para dar una explicación a esto se propone el uso de la explicabilidad, concretamente de la inteligencia artificial explicable (XAI), que ofrece herramientas que permiten interpretar y explicar el comportamiento de manera agnóstica al modelo [4], convirtiendo la 'caja negra' en un proceso transparente.

Aunque los algoritmos bioinspirados sean elegidos principalmente por sus buenos resultados y aplicaciones recientes, carecen de un buen ajuste, que a menudo está hecho por criterio experto, y esto romperá el balance exploración - explotación que tiene el algoritmo; hablamos de exploración como acotamiento al área de soluciones y explotación como búsqueda de estas. Esto antes mencionado compromete enormemente el rendimiento del algoritmo, provocando estancamiento o una convergencia prematura hacia soluciones subóptimas.

En este trabajo de título se buscan abordar los problemas de estancamiento y convergencia prematura, proponiendo un enfoque de explicabilidad post-ejecución. Este método implicará registrar y analizar métricas internas del algoritmo una vez que cada iteración termine. Al examinar indicadores como la densidad poblacional y tasa de mejora, será posible diagnosticar el comportamiento del algoritmo, cómo y cuándo se estancó. Finalmente, el

trabajo busca la implementación de un controlador en línea, el cual guiará la ejecución del algoritmo; este revisará qué operadores/parametrizaciones aportan más progreso en distintos contextos, cuándo conviene ampliar la exploración o cuándo es adecuado modificar la fase de explotación; adicionalmente, este controlador emitirá una explicación de su decisión por iteración.

Este trabajo presenta como principales contribuciones un nuevo enfoque sobre el uso de las metaheurísticas, orientado a su mejora y a la generación de métodos innovadores para la resolución de problemas de optimización computacional, aportando así nuevas herramientas a la comunidad científica.

El documento se estructura de la siguiente manera: Capítulo 2 detalla el marco teórico y el estado del arte, capítulo 3 se enfoca en la definición del problema, solución propuesta, objetivos y metodología; en el capítulo 4 se detallará cómo será la solución y técnicas de análisis.

Capítulo 2

Marco Conceptual y Estado del Arte

2.1. Marco Conceptual

En esta sección se presentarán las definiciones, conceptos y la teoría necesaria para la comprensión del trabajo, así como sus fundamentos.

2.1.1. Heurísticas

Una heurística aplicada a la informática es un método o estrategia que se usa para resolver problemas complejos, tomar decisiones o elegir caminos para ello, aunque no garantice una solución óptima o totalmente exacta. Basada principalmente en reglas empíricas que simplifica la toma de decisiones [5].

2.1.2. Metaheurísticas

Una metaheurística es un marco de trabajo de alto nivel o una estrategia que se utiliza para el diseño de algoritmos de optimización. Su objetivo es explorar de manera eficiente y efectiva un espacio de búsqueda grande y complejo, en el que pueda encontrar soluciones factibles y suficientemente buenas, esto último, no garantiza que sea la solución óptima o exacta. Estas metaheurísticas se utilizan principalmente en problemas del tipo NP-Hard, descritos en la sección 2.1.18. Consta de dos fases claves, las cuales son las siguientes:

- Exploración (Diversificación): Buscar en nuevas áreas del espacio de soluciones para evitar quedarse atrapado en una zona subóptima.
- Explotación (Intensificación): Refinar y mejorar las soluciones prometedoras encontradas en una región específica.

2.1.3. Optimización por enjambre de partículas

La optimización por enjambre de partículas, por sus siglas en inglés (Particle Swarm Optimization), es una **metaheurística** 2.1.2 de optimización global, cuyo fundamento radica en la inteligencia de enjambre, que modela el comportamiento social y colectivo de organismos en la naturaleza, como el vuelo coordinado en bandadas o el movimiento de los cardúmenes. El algoritmo busca la mejor solución a un problema iterativamente a través de la cooperación y la competencia entre sus miembros [6].

El enjambre está compuesto por un conjunto de partículas donde cada una representará una solución potencial al problema propuesto en un espacio multidimensional. Cada una de estas partículas se caracterizará por:

- Posición (X): La ubicación de la partícula en el espacio de búsqueda, que será una potencial solución que está en el instante.
- Velocidad (V): Será la dirección y magnitud del movimiento de la partícula, que determinará cómo se ajustará su posición en la siguiente iteración.

Además, se debe considerar los óptimos globales y locales, los cuales serán:

- Mejor posición individual P_{best} : Será la mejor solución que la partícula ha encontrado a lo largo de la ejecución. (este componente se denomina cognitivo o de experiencia personal)
- Mejor posición global G_{best} : Será la mejor posición que ha sido encontrada por cualquier partícula en todo el enjambre hasta ese momento (este componente se denomina social o de conocimiento colectivo).

Finalmente en cada iteración la velocidad y posición de cada partícula se actualizarán basándose en su inercia w y la influencia de las dos mejores posiciones P_{best} y G_{best} según la siguiente ecuación general:

$$V(t + 1) = wv_i(t) + c_1r_1(P_{best,i} - x_i(t)) + c_2r_2(G_{best} - x_i(t))$$

Donde

- $v_i(t + 1)$: Resultará como la nueva velocidad para la partícula i
- $v_1(t)$: Es la velocidad actual en la partícula i en la iteración actual t .
- $x_i(t)$: Es la posición actual de la partícula i en la iteración actual t .
- c_1 : Es el coeficiente cognitivo, que pondera la experiencia individual
- c_2 : Que es el coeficiente social y pondera la experiencia del enjambre

2.1.4. Algoritmos bio-inspiradas

Los algoritmos o técnicas bio-inspiradas son una categoría de las **metaheurísticas** 2.1.2, que dan resolución a problemas complejos de optimización imitando procesos y comportamientos observados en la naturaleza. Su funcionamiento interno se basa completamente en la inteligencia colectiva de un enjambre o población 2.1.3, para así explorar un determinado espacio de soluciones posibles y converger gradualmente hacia una respuesta o solución suficientemente buena, esto también en un tiempo razonable.

Las aproximaciones que permiten estas técnicas/algoritmos son muy útiles para los problemas clasificados como NP-hard, donde la solución óptima de manera exacta es computacionalmente inviable, ya que el número de posibilidades crecerá exponencialmente de acuerdo con el tamaño del problema.

Estas técnicas van creciendo con el paso del tiempo, y se van creando a medida que se observan fenómenos nuevos [7].

2.1.5. SeaHorse Optimization Algorithm

SeaHorse optimization algorithm, es un algoritmo de optimización metaheurístico 2.1.2 basado en la optimización por enjambre de partículas 2.1.3 el cual se enfoca en el comportamiento de los caballitos de mar [3]. Esta técnica bio-inspirada es la usada en el trabajo. Consta de tres fases claves, las cuales son las siguientes:

1. Movimiento: Define la nueva posición del caballito de mar basado en una inicialización aleatoria.
2. Depredación: Ajusta la posición del caballito de mar, acercándola o alejándola de la **solución élite** con cierta probabilidad de éxito
3. Reproducción: Introducirá nuevos caballitos de mar al enjambre, aprovecha las cualidades de embarazo de los individuos, la cual es la herencia de información positiva.

El concepto de **solución élite** es usado a través de la literatura como un símil de G_{best} o mejor solución global. En las secciones 2.1.6 y 2.1.7 se indicará qué significa y cuáles son los parámetros más importantes del algoritmo.

2.1.6. Exploración - explotación

La exploración o diversificación, es la fase donde la técnica busca en regiones nuevas y desconocidas del espacio de soluciones, mientras que la explotación o intensificación es la fase donde el algoritmo mejora las soluciones actuales para encontrar el óptimo local en dicha región. Mientras que la exploración mantiene movimientos aleatorios por un amplio

espacio de búsqueda aún no evaluadas, la explotación se centra en realizar movimientos más locales o enfocados, examinando de cerca vecinos de las mejores soluciones.

El rendimiento de un algoritmo de optimización recae netamente en la gestión eficiente de estas dos fases, demasiada exploración y el algoritmo saltará constantemente a nuevas áreas sin dedicar suficiente tiempo a mejorar las soluciones que más prometen, demasiada explotación y el algoritmo convergerá rápidamente a un óptimo local y dejará de lado la búsqueda del óptimo global.

Para la técnica bio-inspirada SeaHorse Optimization Algorithm 2.1.5 existe un indicador clave que es la regla de transición, la cual consta de lo siguiente:

$$r = randn(0, 1)$$

Donde r es un número aleatorio, si este número cae sobre el corte derecho $r > 0.5$ se favorecerá la explotación, si este número, por el contrario, cae sobre el corte izquierdo $r \leq 0.5$ se favorecerá la exploración.

2.1.7. Parámetros de ajuste en algoritmos bio-inspirados

Los parámetros de ajuste en metaheurísticas a menudo son referidos a valores internos que toma la técnica para favorecer distintos aspectos. A continuación veremos cómo son los parámetros en el algoritmo SeaHorse detallado en la sección 2.1.5.

1. Levy Flight: Es un tipo de paseo aleatorio en el cual los incrementos son distribuidos de acuerdo a una distribución de probabilidad de cola pesada, en la literatura sobre la técnica SeaHorse [3] se calcula de la siguiente manera:

$$Levy(z) = s \times \frac{w \times \sigma}{|k|^{\frac{1}{\lambda}}}$$

Donde λ es un número entre $[0, 2]$. s es una constante de 0.01, w y k son números aleatorios entre $[0, 1]$ y σ es calculado de manera aparte.

2. Parámetro de predación temporal: Este es un parámetro que actúa como un factor escalador temporal que controla el tamaño del paso del tiempo. Moderando la exploración y explotación en cierta medida. Calculado de la siguiente manera:

$$\alpha = \left(1 - \frac{t}{T}\right)^{\frac{2t}{T}}$$

Donde T es el número de iteraciones máximas y t siendo el número de iteración actual.

Estos parámetros son determinantes e influyentes en los resultados a tener y considerar.

2.1.8. Convergencia prematura y estancamiento

La convergencia prematura y el estancamiento son una contraparte a las ventajas de las técnicas bio-inspiradas [8], es el problema al cual se le quiere dar solución en el trabajo. A continuación se explican en qué consisten:

- Convergencia prematura: Este ocurre cuando el algoritmo pierde rápidamente la diversidad poblacional, y como resultado, todos los individuos se agrupan en torno a un óptimo local en lugar del óptimo global real.
- Estancamiento: Definido como la incapacidad del algoritmo para mejorar el valor de la función objetivo después de un número significativo de iteraciones. Es el síntoma observable de la convergencia prematura.

2.1.9. Hiperheurísticas

Una hiperheurística se define como un marco de trabajo de alto nivel o una estrategia de búsqueda que opera sobre un conjunto de heurísticas o metaheurísticas de bajo nivel (como el algoritmo bio-inspirado en estudio), en lugar de buscar directamente en el espacio de soluciones del problema [9]. Su objetivo fundamental es automatizar y gestionar el proceso de optimización, decidiendo qué técnica o parámetro aplicar en un momento dado para la instancia de un problema específico. Las metaheurísticas base evolucionan hacia sistemas hiperheurísticos cuando su mecanismo de control se enriquece con distintas técnicas que le permiten aprender la mejor acción a tomar, siendo ejemplos de este enriquecimiento el uso de Q-Learning [10] o el Aprendizaje por Refuerzo [11], que son cruciales para un control dinámico y adaptativo del balance exploración-explotación.

2.1.10. Explicabilidad

La explicabilidad en el contexto de la inteligencia artificial se define como el grado de transparencia que permite a un observador humano comprender la causa, la justificación o el proceso que condujo a una predicción o decisión específica del modelo [12]. Su objetivo fundamental es responder a la pregunta fundamental: "¿Por qué se obtuvo este resultado?". Esta cualidad es especialmente crítica en los algoritmos de caja negra, como la optimización por enjambre de partículas 2.1.3, donde la gestión de parámetros de ajuste (ej. w) o el diagnóstico de fallos como el estancamiento no son evidentes. La explicabilidad se convierte así en un requisito esencial para aumentar la fiabilidad, la trazabilidad y la confianza en los sistemas de optimización autónoma.

2.1.11. XAI (Inteligencia Artificial Explicable)

La inteligencia artificial explicable (XAI) es un campo de estudio emergente que se enfoca en el desarrollo de teorías, metodologías y herramientas computacionales para que los sistemas de inteligencia artificial puedan generar las explicaciones necesarias para lograr la explicabilidad [13]. XAI aborda la necesidad de transformar modelos de caja negra en modelos transparentes, que ofrezcan justificación, fidelidad y confianza en sus decisiones. En el presente trabajo, XAI no solo se utiliza para auditar los resultados romperaes, sino que se integra como un módulo de diagnóstico en línea, cuyo propósito es analizar las métricas internas de la metaheurística e inferir, con bases interpretables, el riesgo de estancamiento para justificar la intervención del controlador.

2.1.12. LIME - Local Interpretable Model-agnostic Explanations

LIME o por sus siglas en inglés, Local Interpretable Model-agnostic Explanations, es una técnica de XAI 2.1.11 clasificada dentro de los métodos agnósticos al modelo, lo que le permite generar explicaciones para las predicciones de cualquier modelo de machine learning sin requerir la modificación de su estructura interna. Este método se enfoca en producir explicaciones a nivel local, aproximando el comportamiento de modelos complejos e intrincados mediante el uso de modelos más sencillos e interpretables, como un modelo lineal, en las cercanías de la instancia específica que se está prediciendo. El concepto central de LIME se basa en alterar la instancia de entrada y evaluar cómo dichas perturbaciones impactan la predicción, asignando pesos a las instancias perturbadas según su proximidad al dato original. Gracias a su eficiencia computacional y simplicidad, LIME es adecuado para su uso en escenarios de tiempo real, siendo crucial para proporcionar una visión interna sobre cómo cada característica contribuye a la determinación del modelo para una entrada concreta.

LIME será usado como algoritmo de explicabilidad en este trabajo.

2.1.13. Explicabilidad post ejecución

La explicabilidad post-ejecucion se dará cuando se emite una explicación al cambio del comportamiento del algoritmo tras una iteración de ejecución completa. Esto se encuentra principalmente en hiperheurísticas, donde la salida de una metaheurística es usada como entrada en otra [14]. Es parte del controlador en línea que se desea desarrollar en este trabajo.

2.1.14. Métricas de diagnóstico

Las métricas de diagnóstico son variables internas y continuas que se miden en cada iteración de un algoritmo metaheurístico con el fin de evaluar su estado operacional y su

dinámica de búsqueda. Estas métricas actúan como los indicadores vitales del enjambre y son usadas como entrada principal para el módulo de XAI 2.1.11. Dos métricas críticas para detectar el estancamiento o convergencia prematura son: la diversidad poblacional, que mide la dispersión de las partículas en el espacio de búsqueda (por ejemplo, a través de la varianza o la distancia promedio al centro del enjambre) y cuya caída indica una pérdida de exploración; y la tasa de mejora, que evalúa la frecuencia y magnitud con la que la mejor solución global G_{best} se actualiza.

2.1.15. Programación lineal

La programación lineal es una técnica de optimización ampliamente usada [15] que busca determinar el mejor resultado posible, siendo este el máximo beneficio o el mínimo costo; esto último es aplicado en un modelo matemático donde todos los componentes permanecen lineales. Un problema de programación lineal se caracteriza por una función objetivo, restricciones y variables de decisión; a continuación, se detallarán sus componentes:

- Función objetivo:

$$\text{maximizar/minimizar } f(x)$$

La función objetivo es la que se desea maximizar o minimizar.

- Restricciones: Las restricciones limitan el conjunto de valores permitidos para las variables decisión. Es importante denotar si existen limitaciones de signo

$$x_j \in \mathbb{R}, \forall j \in \{1, \dots, n\}$$

- Variables de decisión: En un modelo de programación lineal, estas variables deben ser capaces de describir completamente las decisiones que puedan ser tomadas

2.1.16. Problemas de optimización computacional

Los Problemas de Optimización Computacional son desafíos en los que se busca encontrar el mejor conjunto de valores para un vector de variables de decisión con el fin de *maximizar/minimizar* $f(x)$ una función objetivo. Estas variables deben satisfacer un conjunto de restricciones que definen el espacio de soluciones factibles.

A diferencia de la Programación Lineal (PL), los problemas de Optimización Computacional presentan grandes diferencias: la función objetivo no necesariamente es lineal, poseen una altísima complejidad computacional (a menudo son NP-Hard), y las regiones de soluciones son notablemente más complejas. Es en esta categoría que se centran los problemas modernos de alta escala [16]. Por la naturaleza de estos problemas, son necesarias las heurísticas y metaheurísticas, que son descritas a continuación en las secciones 2.1.2 y 2.1.1.

2.1.17. Variables discretas y continuas

Las variables se clasifican de acuerdo al tipo de valor que pueden tomar, siendo dos tipos importantes para el desarrollo del presente trabajo:

- Variables discretas: Es aquella que solo puede tomar un número contable de valores dentro de un rango determinado. No se presenta la existencia de intermedios entre dos valores consecutivos. Un ejemplo puede ser el contador de iteraciones en el entorno controlado que se planteará.
- Variables continuas: Son aquellas que pueden tomar un número infinito de valores dentro de cualquier rango especificado. Estos valores se obtienen normalmente tras una medición. Ejemplos de estos valores los encontraremos al momento de realizar análisis estadísticos luego de cada iteración.

Tanto para el análisis estadístico que se quiere realizar, como parámetros de decisión y control, es importante definir de manera correcta estas variables.

2.1.18. Problemas NP-hard

Los problemas NP-Hard (del inglés, Non-deterministic Polynomial-time Hard) son una clase de problemas de optimización y decisión en la teoría de la complejidad computacional que se caracterizan por ser computacionalmente muy difíciles, de los cuales siempre se desean hallar formas de resolverlos de manera más eficiente [17].

2.1.19. Congress on Evolutionary Computation - CEC

La CEC (Congress on Evolutionary Computation) es una conferencia internacional organizada por el IEEE Computational Intelligence Society, donde se presentan avances en optimización, metaheurísticas y algoritmos evolutivos. Dentro de este congreso, se publican periódicamente los CEC Benchmark Functions, que son conjuntos estandarizados de funciones de prueba utilizadas para evaluar y comparar el rendimiento de algoritmos de optimización bajo condiciones controladas (por ejemplo, CEC 2014, CEC 2017, CEC 2022, etc.).

Es importante para el trabajo porque, la suite de funciones de referencia CEC incluye funciones de objetivo parametrizadas por operadores como sesgo, desplazamiento y rotación. La suite CEC representa la plataforma más elaborada para la comparación de algoritmos de búsqueda estocástica [18].

2.1.20. Criterios estadísticos para la evaluación de metaheurísticas

La naturaleza intrínsecamente estocástica de los algoritmos bioinspirados exige que la comparación de rendimiento no se base únicamente en valores promedios. Por el contrario, se debe sustentar en un protocolo estadístico riguroso. Este enfoque es fundamental para discernir si las diferencias observadas en la calidad de las soluciones son estadísticamente significativas o meras variaciones aleatorias.

Para ello, se emplearán pruebas de normalidad (como los tests de *Shapiro-Wilk* [19] y *Lilliefors* [20]) y tests de comparación no paramétricos (como el Test de Wilcoxon [21]) para asegurar la validez de las conclusiones, tal como se detallará en la sección 5.1.

2.1.21. Controlador en línea

Un controlador en línea es un componente de software que interviene y ajusta de forma dinámica los parámetros o la estrategia de un algoritmo durante su ejecución, en tiempo real, a diferencia de los métodos que requieren la pre-configuración de parámetros. En el contexto de las metaheurísticas, el controlador en línea propuesto tiene la función de monitorear activamente las métricas de diagnóstico 2.1.14 del enjambre. Su objetivo es diagnosticar, mediante un módulo de XAI, el riesgo inminente de estancamiento. Una vez diagnosticado, el controlador ejecuta una intervención justificada, ajustando un parámetro clave como el peso de inercia para forzar el cambio de la explotación a la exploración, transformando así la metaheurística de un sistema de parámetros fijos a un sistema adaptativo, auto-ajustable y con trazabilidad.

2.2. Estado del Arte

A continuación, se presenta una revisión de la literatura de carácter sistemático y crítico con un límite temporal estricto de no más de cinco años de antigüedad, cuya metodología detallada se establece en las siguientes secciones. Esta revisión se centrará en cinco ejes de investigación fundamentales:

1. El estado actual de las aplicaciones de metaheurísticas en la resolución de problemas modernos
2. Los trabajos más recientes que emplean el algoritmo Sea-horse Optimizer (SHO)
3. Las mejoras e hibridaciones implementadas en las metaheurísticas para evolucionarlas a sistemas hiperheurísticos
4. La intersección crítica entre las mejoras de metaheurísticas y la Inteligencia Artificial Explicable (XAI)

5. La existencia de trabajos que hayan combinado específicamente el Sea-horse Optimizer y el método LIME, buscando confirmar el vacío de conocimiento que justifica esta propuesta.

Los trabajos son principalmente buscados en base de datos especializadas como, *IEEE Xplore*, *Science Direct*, *Springer*, *ACM Digital Library*, *Web Of Science Scopus*, entre otras.

Finalmente en la siguiente subsección se detallará la metodología a usar 2.2.1.

2.2.1. Metodología, criterios de inclusión y exclusión

Principalmente se usa la nemotecnia *PICO* usada inicialmente en las ciencias de la salud con el objetivo de filtrar información para beneficio de un determinado paciente. Con el paso del tiempo, sufrió adaptaciones y ahora se usa para la búsqueda sistemática enfocada en las investigaciones [22]. Las siglas *PICO* toman el siguiente significado:

- **Población, paciente o problema:** Describe las características principales del grupo de estudio o problema que se desea investigar
- **Intervención:** Define el factor o técnica que se desea evaluar. Se define también como la característica principal del tema escogido.
- **Comparación:** Indica la alternativa a la técnica estándar con la que se comparará la intervención. Nos habla sobre un punto de comparación desde la investigación a un tema parecido en características.
- **Outcomes/Resultado:** Especificará los resultados medibles y esperados, como la mejora que realizó el diagnóstico o tratamiento (en el área médica)

La nemotecnia *PICO* se usará de la siguiente manera en la revisión de la literatura:

- **Población/Problema:** Metaheurísticas Bio-Inspiradas o de Inteligencia de Enjambre (ej., Sea-horse Optimizer - SHO).
- **Intervención:** Un Controlador en Línea basado en XAI (LIME) para ajustar dinámicamente los mecanismos de control de las metaheurísticas.
- **Comparación:** La Metaheurística Original (SeaHorse Optimization Algorithm) con parámetros estáticos o el uso de métodos de control clásicos (rule-based o off-line).
- **Outcomes/Resultados:** Aumento de la trazabilidad (explicabilidad de la decisión), prevención de la convergencia prematura, y mejora en la eficiencia y robustez de la solución romper.

También se hace uso de cadenas de búsqueda estructuradas y específicas, será fundamental para garantizar la exhaustividad y la precisión de la revisión bibliográfica. Estas cadenas emplean operadores booleanos (AND, OR) y truncamientos (*) para mapear los conceptos centrales de la investigación (Metaheurísticas, XAI, SeaHorse Optimization Algorithm, LIME) con sus sinónimos y variaciones. Esto permite maximizar el recobro de literatura relevante en bases de datos detalladas anteriormente, al mismo tiempo que minimiza el ruido generado por documentos tangenciales. También se hace uso de los criterios de inclusión y exclusión, que servirán como filtros que delimitan el alcance de la revisión, asegurando que solo los trabajos que aborden directamente el vacío de conocimiento de la tesis sean seleccionados. Estos últimos elementos se detallan como fueron usados antes de cada revisión.

2.2.2. Aplicaciones de metaheurísticas en la resolución de problemas actuales

Se usa el siguiente string de búsqueda: (**"metaheuristic" OR "swarm intelligence"**) AND (**"real-world application*" OR "complex problem*" OR "industrial optimization" OR "scheduling" OR "feature selection"**). Con los siguientes criterios de inclusión y exclusión:

■ Criterios de inclusión:

- Artículos publicados entre 2020 y 2025 (últimos 5 años).
- Artículos que describan la aplicación de una metaheurística a un problema real, industrial o social (ej., scheduling de producción, optimización de smart grids, feature selection en Big Data, ruteo de vehículos).
- Trabajos que utilicen metaheurísticas bio-inspiradas o de inteligencia de enjambre, como el sea-horse optimizer, PSO, o algoritmos genéticos.

■ Criterios de exclusión:

- Artículos publicados antes del 1 de enero de 2020.
- Trabajos de naturaleza no arbitrada (ej., tesis de grado, pre-prints sin revisión rompera, blogs, white papers).
- Artículos centrados en métodos exactos, métodos de optimización clásica, o algoritmos de machine learning que no se enfoquen en la optimización (ej., clasificación pura).

Dados los criterios y string de búsqueda se dieron los siguientes trabajos:

En el trabajo [23] denotamos el uso de los algoritmos metaheurísticos y cómo se genera una perspectiva integral sobre la aplicación de algoritmos metaheurísticos en los problemas de programación (scheduling) de talleres de producción en el contexto de la Industria 4.0 y la emergente Industria 5.0.

El estudio [24] investiga la aplicación del Algoritmo de Optimización de Orangután, una metaheurística bioinspirada, para resolver problemas de selección de características (Feature Selection). La selección de características es un paso fundamental en el aprendizaje automático para identificar el subconjunto de datos más relevante, lo que mejora el rendimiento del modelo y reduce los costos computacionales.

El siguiente trabajo [25] es una revisión sistemática de la literatura que analiza 161 artículos publicados entre 2019 y 2023 sobre el problema de selección de características (Feature Selection) y su solución mediante metaheurísticas dentro del campo del machine learning.

El objetivo principal de la investigación [26] es realizar un estudio comparativo sobre la aplicación de diecisiete métodos metaheurísticos recientes para optimizar doce problemas distintos de diseño de ingeniería del mundo real.

El trabajo [2] es una revisión de literatura que se centra en las publicaciones recientes (de 2015 a 2021) sobre el despliegue de algoritmos de inteligencia de enjambre (Swarm Intelligence o SI) en la resolución de problemas de programación (scheduling) y optimización en entornos de computación en la nube (Cloud Computing).

El documento [27] describe un estudio que aborda los desafíos que enfrentan los algoritmos de inteligencia de enjambre, como el alto costo computacional y la baja eficiencia, al tratar con datos complejos de alta dimensión en problemas de programación industrial.

El trabajo [28] describe un estudio que aborda los desafíos que enfrentan los algoritmos de Inteligencia de Enjambre, como el alto costo computacional y la baja eficiencia, al tratar con datos complejos de alta dimensión en problemas de programación industrial scheduling. Detalla el uso de las metaheurísticas del algoritmo del lobo gris y el algoritmo del cuco.

El trabajo [1] describe un estudio de revisión que se centra en el uso de algoritmos de optimización metaheurísticos para resolver desafíos en el mundo real, específicamente en las áreas de ingeniería eléctrica y civil.

El trabajo [29] describe un artículo de revisión exhaustivo sobre los avances recientes en los métodos de selección de características, enfocándose en la aplicación de algoritmos metaheurísticos. Esto, para usarlos en problemas de alta dimensionalidad.

El trabajo [30] describe un trabajo de investigación enfocado en mejorar la calidad, rigor científico y transparencia de los estudios que utilizan algoritmos metaheurísticos para la optimización de problemas del mundo real. Este trabajo aporta de manera distinta, ya que no es una revisión del uso de las metaheurísticas sobre cómo son aplicadas.

Como resumen de los trabajos encontrados, se podrán clasificar de la siguiente manera en la tabla 2.1:

Scheuduling	[23] [27] [28] [2] [26]
Feature Selection	[25] [29] [24]
Consideraciones Metodológicas	[1] [30]

Tabla 2.1: Resumen de trabajos encontrados.

Donde:

- Scheuduling: Son metaheurísticas aplicadas a la programación.
- Feature selection: Son metaheurísticas aplicadas a la selección de características
- Consideraciones metodológicas: Son técnicas, investigaciones y revisiones, de como tratar las metaheurísticas para que tengan resultados efectivos, en la aplicación en el mundo real.

2.2.3. Trabajos más recientes con SeaHorse Optimization Algorithm

Se usa el siguiente string de búsqueda: ("Sea-horse optimizer" OR "Sea-horse algorithm" OR "SHO metaheuristic") AND (2023 OR 2024 OR 2025). Con los siguientes criterios de inclusión y exclusión:

- Criterios de inclusión:
 - Artículos publicados entre 2020 y 2025 (últimos 5 años)
 - Artículos que usen en cualquiera de sus formas Sea Horse Optimization algorithm, ya sea, describiendo como se comporta, mejoras o su directo uso.
- Criterios de exclusión:
 - Artículos publicados antes del 1 de enero de 2020.
 - Trabajos de naturaleza no arbitrada (ej., tesis de grado, pre-prints sin revisión rompera, blogs, white papers).
 - Artículos centrados en métodos exactos, métodos de optimización clásica, o algoritmos de machine learning que no se enfoquen en la optimización (ej., clasificación pura).

Dado los criterios y string de búsqueda se encontraron los siguientes trabajos:

El siguiente trabajo [31] presenta el Modified Sea Horse Optimizer (mSHO), una variante mejorada del SeaHorse optimization algorithm (SHO), con el objetivo de resolver problemas complejos de optimización global y de diseño de ingeniería.

El trabajo stockmarket [32] propone un enfoque novedoso para la predicción de precios de acciones, que combina las fortalezas de las redes neuronales de memoria a corto y largo plazo con el algoritmo de optimización de SeaHorse optimization algorithm.

El siguiente artículo [33] propone un método para optimizar automáticamente los hiperparámetros de un modelo de reconocimiento de imágenes agrícolas (basado en ResNet-50), mediante una versión mejorada del algoritmo SeaHorse optimization, que incorpora mapeo caótico y el algoritmo seno-coseno para mejorar su capacidad de búsqueda y estabilidad de convergencia. El algoritmo ESHO optimiza parámetros como tasa de aprendizaje, tamaño de lote y momentum, reduciendo la intervención manual.

Este artículo [34] presenta una versión mejorada del optimizador SeaHorse, denominada SHO-OBL, que incorpora el aprendizaje por oposición durante la fase de depredación y una estrategia de selección ávida al romper cada ciclo para evitar estancamientos en óptimos locales. En comparación con nueve algoritmos metaheurísticos de referencia, el SHO-OBL mostró un desempeño superior en un conjunto de pruebas de optimización global (CEC'20) y en tres problemas de diseño de ingeniería (viga soldada, resorte, recipiente a presión). Además, fue aplicado al problema de selección de cabeceras de clúster en redes de sensores inalámbricos, logrando mejoras en el consumo de energía residual, la vida útil de la red y la duración de la estabilidad. **Este trabajo tiene una importancia significativo para lo que respecta a mejoras en estancamiento en óptimos locales.**

Este artículo [35] propone usar el algoritmo de optimización SeaHorse optimization algorithm para estimar los parámetros del modelo de circuito equivalente de una pila de combustible de membrana de intercambio de protones (PEMFC). El cual demuestra la eficiencia y eficacia del algoritmo de optimización con datos reales.

El trabajo [36] propone un algoritmo híbrido combinación de particle swarm optimization + SeaHorse optimization para resolver el problema de distribución óptima de potencia reactiva, considerando además la integración de vehículos eléctricos en la red. Se realizaron simulaciones en las redes de prueba IEEE de 30 y 57 buses e incluyeron datos reales de vehículos eléctricos para aportar realismo. Los resultados muestran que el algoritmo descrito es eficaz y robusto frente a otros algoritmos metaheurísticos aplicados al mismo problema.

Este artículo [37] titulado propone una versión binaria del optimizador SeaHorse Optimizer, denominada GOG-MBSHO, para seleccionar genes relevantes en datos de expresión génica de cáncer. Para ello, fusiona múltiples estrategias: integración de la estrategia “golden sine” en el movimiento espiral, una estrategia de escape inspirada en el hipopótamo para evitar óptimos locales, y distintas estrategias de pesos de inercia para balancear exploración/explotación. Además, introduce una función de transferencia gaussiana nueva para convertir soluciones continuas a binarias. GOG-MBSHO logra una mejor precisión de clasificación y reduce el número de características seleccionadas en comparación con otros algoritmos binarios de inteligencia de enjambre.

Este estudio [38] presenta el sea Horse optimizer, un nuevo algoritmo de optimización metaheurístico inspirado en el comportamiento de los caballitos de mar, aplicado al control de carga y frecuencia (LFC) en sistemas eléctricos de dos áreas que integran unidades fotovoltaicas y térmicas.

Como resumen de los trabajos encontrados se genera la siguiente tabla:

[31]	Variación y aplicación de la MH.
[33]	Hiperparámetros en imágenes agrícolas.
[34]	Integra pruebas en redes de sensores
[35]	Estimación en celdas de combustible.
[36]	Combina la MH con PSO, para optimizar la distribución de potencia, en autos eléctricos.
[37]	Identificación de genes de cáncer.
[38]	Optimizar controladores en sistemas eléctricos

Tabla 2.2: Resumen de trabajos aplicando SHO

2.2.4. Mejoras realizadas a las metaheurísticas

Se usa el siguiente string de búsqueda: (**"adaptive metaheuristic" OR "self-adaptive metaheuristic" OR "hyper-heuristic" OR "dynamic parameter control"**) AND (**"convergence detection" OR "stagnation avoidance" OR "exploitation exploration balance"**). Con los siguientes criterios de inclusión y exclusión:

- Criterios de inclusión:
 - Artículos publicados entre 2020 y 2025 (últimos 5 años)
 - Artículos que mejoren de cualquier manera los algoritmos metaheurísticos, en cualquiera de sus modalidades.
- Criterios de exclusión:
 - Artículos publicados antes del 1 de enero de 2020.
 - Trabajos de naturaleza no arbitrada (ej., tesis de grado, pre-prints sin revisión rompera, blogs, white papers).

Dado los criterios y string de búsqueda se encontraron los siguientes trabajos:

Este trabajo [8] aborda el problema del estancamiento en algoritmos metaheurísticos mediante una estrategia de nivel meta llamada MsMA. MsMA actúa externamente al algoritmo base, monitoreando su desempeño para detectar estancamientos, y cuando esto ocurre, divide el cómputo restante en particiones para reiniciar el algoritmo en cada segmento,

guiando las nuevas ejecuciones según el historial de soluciones. El enfoque no requiere modificar la lógica interna del algoritmo metaheurístico.

Este estudio [39] presenta una versión mejorada del algoritmo beluga whale optimization (BWO), denominada mBWO, diseñada para abordar problemas de optimización global y de ingeniería. El mBWO incorpora tres estrategias clave: Estrategia de evolución de élite: conserva las mejores soluciones para las generaciones siguientes, evitando la convergencia prematura. Mutación aleatoria controlada: introduce variaciones aleatorias para mejorar la exploración del espacio de búsqueda. Factor de transición entre exploración y explotación: ajusta dinámicamente el equilibrio entre exploración y explotación durante la ejecución del algoritmo.

Este trabajo [40] propone una mejora en las hiperheurísticas para problemas de optimización combinatoria (COPs), integrando un modelo de Multi-Armed Bandit (MAB) con el algoritmo EXP3 para equilibrar exploración y explotación. La metodología se aplica al Problema de Ruteo de Vehículos con Ventanas de Tiempo (VRPTW), utilizando una lista diversa de heurísticas de bajo nivel.

Este estudio [41] presenta un algoritmo híbrido denominado HBASCSO, que combina el Honey Badger Algorithm (HBA) y el Sand Cat Swarm Optimization (SCSO) para mejorar la eficiencia en la resolución de problemas de optimización global.

Este estudio [42] propone un algoritmo de evolución diferencial adaptativa (ADE) que incorpora una estrategia de evolución adaptativa y un mecanismo de mejora de la diversidad para abordar problemas de optimización complejos. La estrategia adaptativa ajusta dinámicamente los parámetros del algoritmo durante la ejecución, mientras que el mecanismo de mejora de la diversidad mantiene la diversidad de la población para evitar la convergencia prematura capacidad de exploración.

Este artículo [43] proporciona una revisión exhaustiva de las hiperheurísticas, técnicas de búsqueda diseñadas para seleccionar, generar y secuenciar metaheurísticas con el fin de resolver problemas de optimización complejos. Se presenta una taxonomía detallada que clasifica las hiperheurísticas según diversos criterios, como el tipo de problemas que abordan, las técnicas de aprendizaje que emplean y su capacidad de adaptación. Nos ayuda a entender las diferencias entre metaheurísticas mejoradas.

El artículo [44] presenta EvoMapX, un marco explicativo diseñado para hacer transparentes las dinámicas internas de los algoritmos de optimización basados en poblaciones, comúnmente utilizados en problemas NP-hard y de alta dimensión. El sistema incorpora tres estructuras interpretables: la Operator Attribution Matrix (OAM), que mide la contribución de cada operador; el Population Evolution Graph (PEG), que visualiza la genealogía y evolución de las soluciones; y el Convergence Driver Score (CDS), que identifica los operadores que impulsan la convergencia. Evaluado en cuatro POAs con el conjunto de pruebas CEC 2021, EvoMapX demostró ofrecer explicaciones gráficas y textuales valiosas sobre el comportamiento de los algoritmos, fomentando una mayor interpretabilidad y confianza en la optimización metaheurística.

El estudio [45] propone el Hierarchically Controlled Differential Evolution (HCDE), una versión mejorada del algoritmo Differential Evolution (DE) diseñada para superar sus principales limitaciones, como la baja velocidad de convergencia y la tendencia a atascarse en óptimos locales al abordar problemas complejos. HCDE introduce una estrategia de control jerárquico que optimiza tres aspectos clave: el ajuste dinámico de parámetros mediante distribuciones logística y Cauchy, una estructura de archivo multinivel que combina soluciones élite y no élite para equilibrar exploración y diversidad, y un mecanismo basado en entropía que mide la diversidad poblacional y activa perturbaciones híbridas y reinicios cuando se detecta estancamiento. Probado en 88 funciones de referencia de los conjuntos CEC2014, CEC2017 y CEC2022, y comparado con siete variantes de DE de última generación, HCDE mostró mejoras significativas en precisión, velocidad de convergencia y robustez, además de validar su eficacia en una aplicación de optimización estructural de puentes. **Este estudio, es de suma importancia para el trabajo por la manipulación de parámetros.**

El artículo [46] propone un enfoque hiperheurístico denominado Hyper-heuristic Adaptive Simulated Annealing with Reinforcement Learning (HHASA) para abordar el problema de ruteo de vehículos eléctricos con capacidad limitada (CEVRP), enfocado en optimizar la logística de última milla en entornos urbanos y así contribuir a la sostenibilidad social y económica. HHASA combina un método de multi-armed bandit con el algoritmo Simulated Annealing auto-adaptativo, permitiendo ajustar dinámicamente las estrategias de búsqueda mediante aprendizaje por refuerzo. Este enfoque considera las restricciones reales de los vehículos eléctricos, como la disponibilidad de estaciones de carga y la autonomía limitada, para planificar recargas anticipadas y minimizar tiempos y costos de recorrido. Los resultados experimentales muestran que HHASA mejora múltiples soluciones óptimas conocidas y logra los mejores valores promedio en instancias de alta dimensión del conjunto de pruebas propuesto para la competencia IEEE WCCI 2020, destacando su eficacia en problemas complejos de optimización logística sostenible.

El trabajo [47] presenta una revisión exhaustiva del algoritmo Differential Evolution (DE), destacando los avances más recientes entre 2019 y 2023 relacionados con el equilibrio entre exploración y explotación, un aspecto clave para su desempeño en optimización global. La revisión clasifica las estrategias de mejora del DE en tres niveles: a nivel de algoritmo, donde se analizan los métodos de hibridación con otras metaheurísticas; a nivel de operadores evolutivos, donde se discuten operadores mejorados y estrategias de selección adaptativa; y a nivel de parámetros, donde se resumen distintos enfoques de control y adaptación dinámica. En conjunto, el trabajo ofrece una actualización sistemática y estructurada del estado del arte sobre cómo los investigadores han abordado el balance exploración–explotación en DE, proporcionando una visión integral para futuras investigaciones y desarrollos del algoritmo.

Como resumen de los trabajos encontrados se genera la siguiente tabla, destacando la mejora a la metaheurística, transformándola en hiperheurística.

[8]	Propone MsMA, menjando estancamientos.
[39]	Introduce una mejora a una MH, con evolución y ajuste dinámica.
[40]	Presenta una hiperheurística hibridando dos algoritmos
[41]	Propone HBASCSO que utiliza dos MH's.
[42]	Desarrolla ADE, con ajuste dinámico.
[43]	Revisión exhaustiva de las hiperheurísticas.
[44]	Presenta EvoMapX marco explicativo para las heurísticas.
[45]	Propone HCDE, una versión jerárquica con modificación de parámetros
[46]	Propone HHASA, hiperheurística con aprendizaje de refuerzo
[47]	Revisión actualizada de Differential Evolution, mejorándolo.

Tabla 2.3: Resumen de trabajos hiperheurísticos

2.2.5. Mejoras realizadas a las metaheurísticas, con explicabilidad

Se usa el siguiente string de búsqueda: ("XAI" OR "explainable artificial intelligence" OR "interpretable machine learning") AND ("metaheuristic" OR "optimization algorithm") AND ("parameter tuning" OR "diagnosis"). Con los siguientes criterios de inclusión y exclusión:

- Criterios de inclusión:
 - Artículos publicados entre 2020 y 2025 (últimos 5 años)
 - Artículos que hibride de cualquier manera los algoritmos metaheurísticos, con la inteligencia artificial explicable.
 - Uso de XAI para diagnosticar o ajustar algoritmos metaheurísticos.
- Criterios de exclusión:
 - Artículos publicados antes del 1 de enero de 2020.
 - Trabajos de naturaleza no arbitrada (ej., tesis de grado, pre-prints sin revisión rompera, blogs, white papers).

Dado los criterios y string de búsqueda se encuentran los siguientes trabajos:

Este estudio [48] propone un enfoque de predicción de parámetros para algoritmos metaheurísticos aplicados al Problema de Ruteo de Vehículos con Ventanas de Tiempo (CVRPTW), utilizando aprendizaje automático para ajustar dinámicamente los parámetros del algoritmo objetivo, mejorando la eficiencia computacional y la precisión de la solución. Usa XAI y algoritmos de explicabilidad para el análisis de resultados.

Este estudio [49] propone un modelo interpretable para el diagnóstico de fallas en transformadores sumergidos en aceite, diseñado para implementarse en dispositivos edge y superar problemas de "caja negra" y seguridad de datos. Primero, se introduce un

algoritmo de extracción de características basado en teoría de conjuntos difusos en dominio real, que mejora el rendimiento del modelo y reduce el tiempo de inferencia. Luego, los hiperparámetros de Extreme Gradient Boosting se optimizan automáticamente con el método de Newton–Raphson, logrando una mayor precisión frente a otros métodos. Finalmente, se emplea SHAP para interpretar el impacto de cada gas disuelto en los resultados de diagnóstico, y se valida la robustez, confiabilidad e interpretabilidad del modelo mediante casos reales, ofreciendo insights prácticos para la operación y mantenimiento de transformadores.

Este trabajo [50] propone un modelo de aprendizaje automático en ensamblaje robusto e interpretable para la detección de fraudes en seguros de salud, abordando desafíos como la alta dimensionalidad de los datos y la falta de transparencia en modelos complejos. El enfoque se divide en tres etapas: preprocesamiento de datos, entrenamiento e integración de modelos, y análisis de resultados con interpretación de características. Se emplearon métodos de selección de características como Embedded y Permutation para reducir la dimensionalidad sin sacrificar el rendimiento. Posteriormente, se aplicaron técnicas de ensamblaje como Voting, Weighted y Stacking para combinar diversos modelos y evaluar su desempeño. La interpretación de las características se realizó mediante herramientas como Partial Dependence Plots (PDP), SHAP y LIME, permitiendo comprender el impacto de cada variable en las predicciones. Los resultados demostraron una mejora en la precisión de la detección de fraudes y proporcionaron información valiosa sobre la interpretabilidad de los modelos de aprendizaje automático en este contexto. Usa modelos metaheurísticos como el Puma Optimizer.

Este estudio [4] presenta una revisión sistemática que analiza la necesidad de explicabilidad en métodos de inteligencia computacional para optimización, como metaheurísticas, incluyendo algoritmos evolutivos e inteligencia de enjambre. Se enfoca en cuatro aspectos: la contribución de métodos de XAI (como SHAP y LIME) para interpretar el desempeño de las metaheurísticas; la influencia de las características del problema en el comportamiento de búsqueda y la explicabilidad; el papel de la teoría matemática para ofrecer explicaciones transparentes; y el potencial de las metaheurísticas para mejorar la interpretabilidad de modelos de IA, como los de machine learning. Además, se introducen conceptos como análisis de paisaje y diversidad de calidad para evaluar algoritmos en distintos escenarios, y se propone “Explainergy”, un enfoque innovador para integrar la explicabilidad en algoritmos metaheurísticos en el ámbito energético, aumentando la transparencia y utilidad de los modelos de optimización. Esta revisión sirve como base para futuras investigaciones que combinen explicabilidad con computación evolutiva y optimización metaheurística para resolver problemas del mundo real en diversos campos. **Este trabajo presenta un aporte inmenso a la revisión de la literatura.**

Este artículo [51] presenta una revisión exhaustiva sobre cómo el aprendizaje automático (ML) puede potenciar las metaheurísticas en la optimización global, especialmente en problemas complejos donde los métodos clásicos enfrentan limitaciones. Se exploran

cómo técnicas de machine learning como el aprendizaje por refuerzo, redes neuronales y modelos de sustitución pueden mejorar la eficiencia y adaptabilidad de algoritmos como particle swarm optimization, algoritmos evolutivos y algoritmos de enjambre. Se discuten aplicaciones en áreas como la energía, la logística y la ingeniería, donde la combinación de ML y metaheurísticas ha mostrado resultados prometedores.

Este estudio [52] presenta Xtune, un método innovador y eficiente para la optimización de hiperparámetros en modelos de pronóstico de series temporales basados en aprendizaje profundo. A diferencia de enfoques tradicionales como la búsqueda en cuadrícula o aleatoria, que requieren evaluaciones exhaustivas y son computacionalmente costosos, Xtune utiliza técnicas de Inteligencia Artificial Explicable (XAI), específicamente SHAP, para determinar el tamaño óptimo de la ventana deslizante en el análisis de series temporales. Los experimentos realizados con conjuntos de datos reales muestran que Xtune identifica de manera eficiente el valor óptimo del hiperparámetro y supera a los métodos existentes en términos de velocidad de ejecución. Además, Xtune se aplica con éxito en la detección de anomalías mediante el método SALAD, demostrando su versatilidad y eficacia en tareas de pronóstico y detección de anomalías en series temporales. Mencionando la posibilidad del uso del ajuste de los hiperparámetros en algoritmos de optimización, es por este motivo que este artículo se encuentra en la sección, ya que no involucra ninguna metaheurística.

Este estudio [53] propone el modelo LXAI OA-ADPCM, que integra técnicas de inteligencia artificial explicativa (XAI) con algoritmos de optimización para la predicción y clasificación de la demencia, mejorando los sistemas de soporte a la decisión clínica. El modelo emplea normalización de datos, selección de características mediante el algoritmo Naked Mole-Rat (NMRA), y combina clasificadores como BiLSTM, autoencoders dispersos (SAE) y redes neuronales convolucionales temporales (TCN). La optimización de hiperparámetros se realiza mediante el algoritmo de optimización Gazelle (GOA), y la técnica Grad-CAM se utiliza para proporcionar interpretaciones visuales de las decisiones del modelo.

Este estudio [54] presenta un modelo híbrido denominado RBMO-ELM (Red-Billed Blue Magpie Optimizer-tuned Extreme Learning Machine), diseñado para predecir las emisiones de CO_2 en economías en desarrollo, donde existen limitaciones de datos y capacidades técnicas. El modelo combina el algoritmo de optimización RBMO, inspirado en el comportamiento de forrajeo del magpie de pico rojo, con la red neuronal de aprendizaje extremo (ELM). El RBMO ajusta eficientemente los pesos y sesgos de la capa oculta de la ELM, mejorando la estabilidad y el poder predictivo del modelo. En este trabajo se usa la explicabilidad en conjunto a las técnicas de aprendizaje reforzado.

Como resumen de los trabajos encontrados, se genera la siguiente tabla, destacando las características más relevantes.

[48]	Modelo predictico.
[49]	Modelo interpretable, SHAP.
[50]	LIME y SHAP, MH de puma.
[4]	Revisión sistemática.
[51]	Mejorar transparencia y trazabilidad
[52]	Presenta ajuste hiperparámetros con XAI.
[53]	Propone modelo con XAI.
[54]	Introduce XAI mediante machine learning a la MH.

Tabla 2.4: Resumen metaheurísticas y explicabilidad.

2.2.6. SeaHorse Optimization Algorithm y LIME

Se usa el siguiente string de búsqueda: (("Sea-horse optimizer" OR "SHO algorithm") AND ("LIME" OR "SHAP" OR "XAI")). Con los siguientes criterios de inclusión y exclusión:

- Criterios de inclusión:
 - Artículos publicados entre 2020 y 2025 (últimos 5 años)
 - Artículos que de manera excluyente usen LIME.
 - Artículos que de manera excluyente usen SHO.
- Criterios de exclusión:
 - Artículos publicados antes del 1 de enero de 2020.
 - Trabajos de naturaleza no arbitrada (ej., tesis de grado, pre-prints sin revisión romper, blogs, white papers).

Solo fue encontrado un artículo [55], el cual usa SeaHorse Optimization Algorithm en conjunto con SHAP values y LIME, en la trama de explicabilidad. Pero solo lo hace a modo de análisis de resultados, no como se espera para este trabajo, que es, guiando el algoritmo en alguna de sus fases. El trabajo consiste en el análisis de rayos-x para el diagnóstico de numonía, usando las técnicas antes mencionadas.

Capítulo 3

Definición del Problema

3.1. Formulación del Problema

A pesar de los recientes avances en los análisis post-ejecución, que permiten auditar el comportamiento del algoritmo una vez finalizada la búsqueda, estos no son efectivos para guiar la exploración en tiempo real, como denotamos en la revisión de la literatura. Esta limitación es la raíz de problemas recurrentes como el estancamiento del algoritmo en óptimos locales, la excesiva aleatoriedad y la convergencia prematura, finalmente dando lugar a hallar soluciones subóptimas.

3.2. Solución Propuesta

Para abordar los problemas de estancamiento y convergencia prematura, se propone un enfoque de explicabilidad post-ejecución. Este método consiste en registrar y analizar las métricas internas del algoritmo una vez finalizada su ejecución. Al examinar indicadores como la densidad poblacional y la tasa de mejora, es posible diagnosticar el comportamiento del algoritmo, identificando los momentos y las causas de un posible estancamiento.

El objetivo de esta investigación es diseñar e implementar un controlador interpretable que supervise indicadores clave de desempeño —como la diversidad poblacional, la tasa de mejora y el éxito por operador— y que, además, genere explicaciones por iteración. Este diseño permitirá anticipar estancamientos, determinar cuándo conviene ampliar la exploración y reconocer qué operadores o parametrizaciones aportan mayor progreso en distintos contextos, proporcionando así trazabilidad y consistencia, superando las limitaciones de los enfoques puramente post-ejecución.

La solución propuesta busca guiar la búsqueda en tiempo real de forma retrospectiva, aportando transparencia y coherencia al proceso de optimización. De esta manera, se podrá

identificar con claridad qué operadores o configuraciones contribuyeron de forma más significativa al avance del algoritmo, ofreciendo la justificación necesaria para realizar ajustes informados en futuras ejecuciones.

Considerando los hallazgos presentados en la sección 2.2, la revisión de la literatura muestra que la explicabilidad se ha empleado principalmente para diagnosticar el funcionamiento del algoritmo a modo de análisis posterior. En contraste, este trabajo propone emplear la explicabilidad como un mecanismo activo de guía dentro de la metaheurística seleccionada, potenciando su capacidad adaptativa y su comprensión en tiempo de ejecución.

3.3. Objetivos

3.3.1. Objetivo General

Desarrollar un controlador interno aplicando algoritmos de explicabilidad in-ejecución a metaheurísticas bio-inspiradas generando un componente de explicabilidad, para evitar estancamientos, convergencia prematura y soluciones sub-óptimas.

3.3.2. Objetivos Específicos

1. Revisión bibliográfica crítica y sistemática sobre la integración de metaheurísticas con técnicas de explicabilidad.
2. Implementación de algoritmo metaheurístico bio-inspirado, identificando parámetros críticos y testeó en escenarios de prueba validados.
3. Desarrollar un método de explicabilidad que tenga trazabilidad sobre el comportamiento de la metaheurística aplicada.
4. Integración de la metaheurística seleccionada junto con la técnica de explicabilidad. Para finalmente evaluar desempeño, predecir estancamientos y prever la convergencia prematura
5. Aplicación benchmarks que validen la propuesta.

3.4. Metodología

Debido a la naturaleza de la asignatura Proyecto de Título se basa en concentrar la propuesta y buscar una línea en la que se pueda desarrollar/investigar se decidió por la

siguiente metodología, research design del tipo experimental [56], la cual ayuda en planificar el 'cómo' del trabajo asegurando que los resultados sean válidos y fiables. Consta de lo siguiente (se puede visualizar en la Figura 3.1):

1. Formular hipótesis de investigación: preguntarse que sucederá con la implementación de un controlador en línea
2. Identificar variables:
 - a) Variable independiente: La que se manipulará, en este caso, la existencia de la implementación descrita en el punto 3.2.
 - b) Variables dependientes: Las que se definirán para medir el efecto, se deberá tener cautela con la medición y definición de los parámetros que afectan nuestra metaheurísticas.
 - c) Variables de control: Para garantizar la fiabilidad de los resultados, se establecen variables de control que permanecen constantes. Entre ellas se encuentran los escenarios de las bibliotecas digitales, la metaheurística, el número de iteraciones, los benchmarks provistos por la CEC 2.1.19 (a método de validación) y otros factores que definen el espacio muestral.
3. Seleccionar el diseño experimental: Diseñar un grupo de control y un grupo experimental
4. Preparar los escenarios de prueba: Definir los datos/escenarios que se usarán para los experimentos, un ejemplo válido serán los problemas de última milla.
5. Realizar los benchmarks provistos por la CEC.
6. Planificar el análisis de datos: Una vez teniendo los resultados se planea hacer un análisis estadístico inferencial.

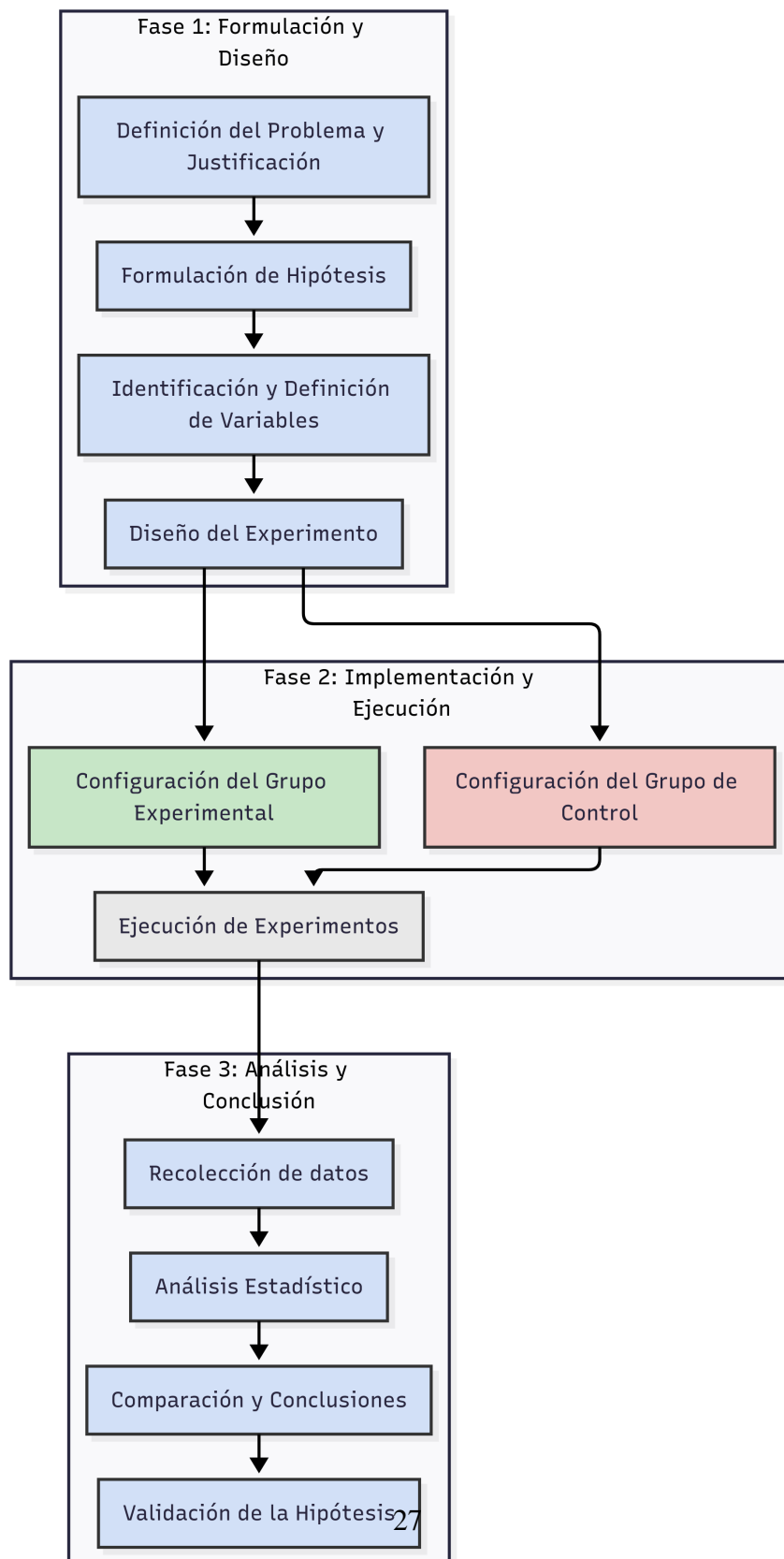


Figura 3.1: Metodología ilustrada

Capítulo 4

Diseño de la solución

4.1. Solución del problema

Para el diseño de la solución —basada en el controlador en línea descrito en el Capítulo 1— se realizó un desglose progresivo desde los conceptos generales hasta el funcionamiento operativo del sistema, estructurado en los siguientes puntos:

1. Arquitectura y funcionamiento del controlador.
2. Métricas a monitorear.
3. Módulo XAI.
4. Ciclo de control.
5. Validación del controlador.

4.1.1. Arquitectura y funcionamiento del controlador

En su forma base, una metaheurística funciona como un algoritmo optimizador que, a partir de un escenario y un conjunto de hiperparámetros definidos, busca una solución óptima. En la Figura 4.1 se representa dicho proceso: el escenario define el problema a resolver (ya sea un caso real o una función benchmark), la metaheurística actúa como motor de búsqueda, y los hiperparámetros son ajustados por criterio experto para guiar el desempeño del algoritmo. Estos elementos conforman el flujo tradicional de resolución de problemas mediante metaheurísticas. Sin embargo, este esquema carece de mecanismos de supervisión o interpretación durante la ejecución, lo que limita su capacidad adaptativa.

Sin embargo, este enfoque presenta limitaciones recurrentes, tales como el estancamiento, la convergencia prematura y la falta de trazabilidad en la toma de decisiones del

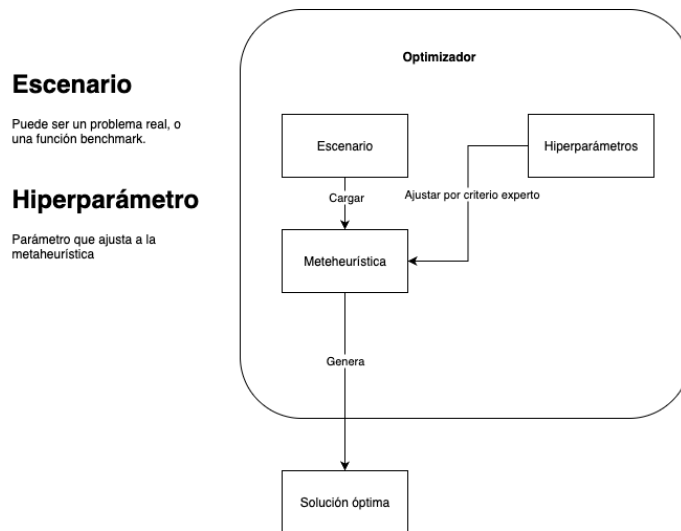


Figura 4.1: Funcionamiento base de una metaheurística.

algoritmo. Para abordar estas limitaciones, se propone incorporar un controlador en línea, cuya arquitectura se muestra en la Figura 4.2.

El controlador en línea actúa como un módulo supervisor que opera de manera sincrónica con el optimizador base. Este controlador integra un módulo explicable (XAI) que observa, verifica y analiza el comportamiento de la metaheurística en tiempo real. A partir de métricas internas —como la tasa de mejora, la diversidad poblacional y los indicadores de éxito por operador— el módulo XAI identifica posibles estancamientos y emite recomendaciones o ajustes sobre los hiperparámetros, retroalimentando continuamente al optimizador.

Este ciclo de observación, análisis y ajuste permite que la búsqueda sea guiada de forma dinámica, generando trazabilidad y justificaciones interpretables de las decisiones tomadas. Como resultado, el controlador no solo mejora la adaptabilidad del proceso de optimización, sino que también fortalece su transparencia y coherencia metodológica. Los registros generados por el módulo XAI se emplearán posteriormente en un análisis estadístico inferencial para evaluar la efectividad del controlador.

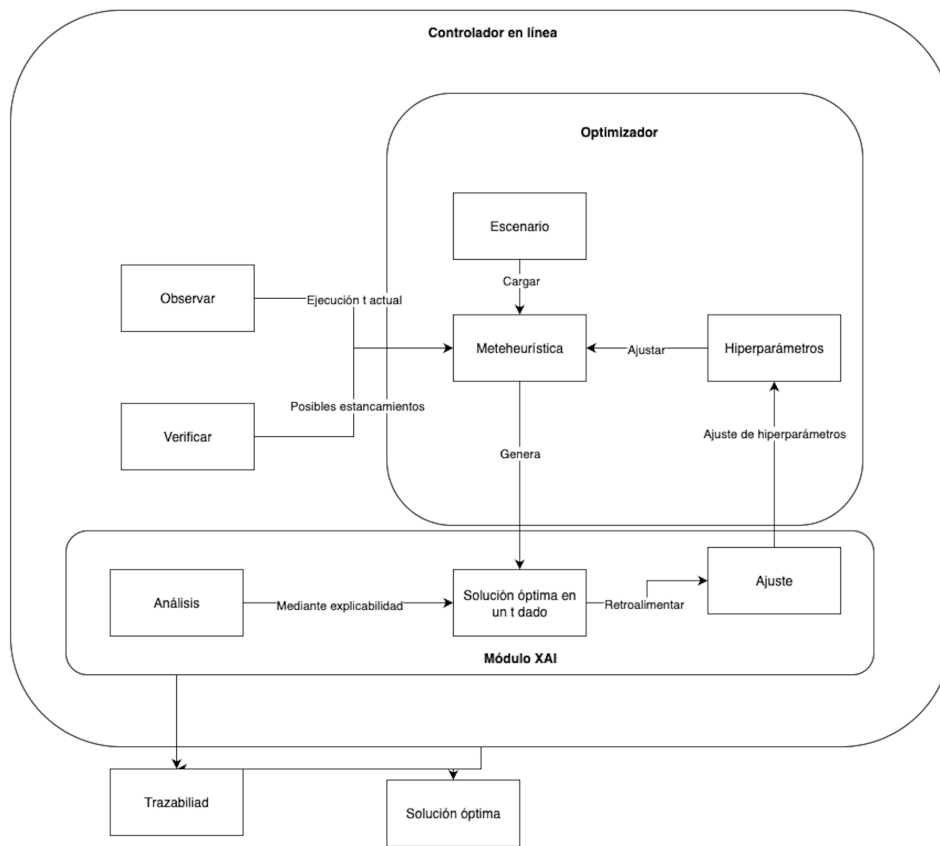


Figura 4.2: Arquitectura general del controlador en línea propuesto.

4.1.2. Métricas a monitorear

Para el análisis en tiempo de ejecución, el controlador recopila métricas como la densidad poblacional, la tasa de mejora y el éxito por operador. Estas variables permiten evaluar en tiempo real la dinámica evolutiva del algoritmo, proporcionando señales cuantitativas para la generación de explicaciones y decisiones adaptativas. Las variables a considerar son:

- Densidad poblacional: Consta de la varianza o distancia media entre individuos, esta métrica nos advierte de una convergencia prematura.
- Tasa de mejora: Proveniente de $\Delta Fitness$ entre iteraciones consecutivas, lo cual, detecta la progresión entre soluciones nuevas encontradas, nos permitirá detectar estancamientos.
- Variables que surgen de las mejoras obtenidas por cada operador (estrategia aplicada por la metaheurística), permitiendo evaluar su efectividad local.

- Entropía poblacional: Mide la dispersión de soluciones, aspecto clave para detectar pérdidas de variabilidad.
- Velocidad de convergencia: Representa la pendiente de mejora del valor de *Fitness*, utilizada para medir la estabilidad del algoritmo.

4.1.3. Módulo XAI

El mecanismo explicable constituye el núcleo analítico del controlador en línea, responsable de interpretar el comportamiento del algoritmo y justificar las acciones de ajuste en tiempo real. Para ello, se integran técnicas de Inteligencia Artificial Explicable (XAI), concretamente LIME, la cual permite cuantificar la influencia de cada métrica monitoreada sobre las decisiones del controlador.

Durante la ejecución, el módulo explicable genera representaciones interpretables —como mapas de impacto o gráficos de contribución por métrica— que describen cómo las variaciones en el comportamiento del algoritmo afectan su rendimiento global. De esta forma, cada decisión de ajuste (como modificar un hiperparámetro o cambiar la probabilidad de un operador) puede ser rastreada y justificada mediante una explicación legible.

Así, el controlador actúa no solo como un sistema adaptativo, sino también como un sistema interpretable y transparente, proporcionando trazabilidad a cada decisión de optimización.

4.1.4. Ciclo de control

El ciclo de control define la secuencia de operaciones que permiten al sistema supervisar, analizar y ajustar la metaheurística de forma continua. El proceso se desarrolla de acuerdo con las siguientes etapas:

1. Observación: en cada iteración t , el controlador recopila métricas internas del optimizador.
2. Verificación: se evalúan las condiciones de desempeño del algoritmo, identificando posibles estancamientos o pérdida de diversidad.
3. Análisis mediante explicabilidad: el módulo XAI analiza las métricas recolectadas, determinando la relevancia de cada una en la pérdida o mejora de rendimiento.
4. Ajuste: con base en las explicaciones obtenidas, se modifican los hiperparámetros o estrategias activas de la metaheurística.
5. Retroalimentación: las nuevas configuraciones se aplican al optimizador, reiniciando el ciclo con los parámetros actualizados.

El ciclo continúa hasta alcanzar el criterio de parada definido. De este modo, el controlador mantiene una interacción constante entre observación, análisis y acción, garantizando una búsqueda guiada, trazable y coherente.

4.1.5. Validación del controlador

La validación del controlador en línea se llevará a cabo mediante un proceso experimental comparativo, siguiendo las directrices establecidas por el CEC 2.1.19. Para ello, se emplearán las funciones de referencia provistas por este organismo, permitiendo realizar pruebas de benchmark que respalden o refuten los resultados obtenidos al incorporar el controlador en la metaheurística base. Este proceso de validación constituye el punto de partida para evaluar la eficiencia, robustez y capacidad adaptativa del algoritmo base frente a su versión controlada.

4.2. Técnicas de análisis

El análisis de los resultados obtenidos mediante la implementación del controlador en línea se realizará mediante un enfoque combinado que integra técnicas de análisis estadístico inferencial y evaluaciones comparativas basadas en benchmarks. El objetivo de este proceso es medir de forma objetiva la eficacia, estabilidad y robustez del sistema propuesto frente a la metaheurística base, determinando si las diferencias observadas en el rendimiento son estadísticamente significativas y no producto del azar.

Por una parte, el análisis inferencial permitirá contrastar el comportamiento de ambas versiones del algoritmo mediante pruebas de significancia. En particular, se empleará la prueba no paramétrica de *Wilcoxon-Mann-Whitney* [57], adecuada para comparar distribuciones independientes sin asumir normalidad en los datos. Esta prueba se aplicará sobre métricas clave como el valor final de la función objetivo, la tasa de convergencia y la diversidad poblacional media.

Las hipótesis estadísticas que guiarán este análisis serán las siguientes:

- Hipótesis nula (H_0): No existen diferencias estadísticamente significativas entre el desempeño de la metaheurística base y la versión con el controlador en línea. Cualquier variación observada en los resultados se debe al azar.
- Hipótesis alternativa (H_1) : Existen diferencias estadísticamente significativas entre ambos enfoques, lo que indica que el controlador en línea produce una mejora real en el desempeño de la metaheurística

Por otra parte, el análisis de benchmarks permitirá una comparación directa del desempeño del controlador utilizando conjuntos de funciones de prueba estandarizadas y ampliamente reconocidas en la comunidad científica. En este caso, se emplearán las funciones

provistas por la CEC, las cuales ofrecen escenarios controlados y reproducibles que facilitan la evaluación comparativa entre algoritmos.

Los datos se obtendrán a partir de múltiples ejecuciones independientes de la meta-heurística, tanto con como sin el controlador en línea, utilizando las funciones de referencia propuestas por el CEC. En cada ejecución se registrarán indicadores como el valor final de la función objetivo (fitness), el número de iteraciones hasta la convergencia, la diversidad poblacional media, la tasa de mejora promedio y el éxito por operador. Cada experimento se repetirá un número suficiente de veces —por ejemplo, 30 ejecuciones— para garantizar la validez estadística y minimizar la influencia de la aleatoriedad inherente a las meta-heurísticas.

Los resultados serán almacenados en archivos estructurados en formato CSV o JSON, con identificadores únicos por ejecución, función de prueba y configuración experimental.

Los resultados se representarán mediante gráficos comparativos que faciliten la interpretación visual del desempeño del sistema. Se utilizarán diagramas de caja para observar la distribución de los valores de fitness, curvas de convergencia que muestren la evolución del rendimiento a lo largo de las iteraciones, y mapas de calor para analizar correlaciones entre métricas internas. Asimismo, se incluirán visualizaciones generadas por el módulo explicable, como mapas de importancia de variables o gráficos de contribución de métricas según LIME, con el fin de fortalecer la interpretabilidad de los resultados.

Capítulo 5

Experimentación

5.1. Diseño de experimentos

5.1.1. Diseño para la etapa de experimentación

A continuación, se presenta en la Figura 5.1 un diagrama que detalla el diseño experimental propuesto a alto nivel. El proceso metodológico establece que, para cada instancia de problema —ya sea una función de benchmark estandarizada (v.g., el conjunto de funciones CEC) o un problema de optimización aplicada (v.g., el ruteo de última milla—, el algoritmo será evaluado mediante N ejecuciones independientes. Considerar que se usa la palabra **hibridación** en referencia a la metaheurística mejorada con el controlador en línea.

La realización de este conjunto de N ejecuciones independientes es un requisito fundamental para garantizar la robustez estadística y la fiabilidad de los resultados del algoritmo. A partir de los datos recolectados, se aplicará un análisis estadístico inferencial riguroso. Este análisis permitirá extraer métricas de rendimiento clave, tales como la media, la mediana y la desviación estándar, proporcionando una base empírica sólida para la comparación y validación del algoritmo propuesto. Este enfoque cumple, a su vez, con los estándares de validación establecidos por las funciones benchmark de la CEC (fundamental en la literatura).

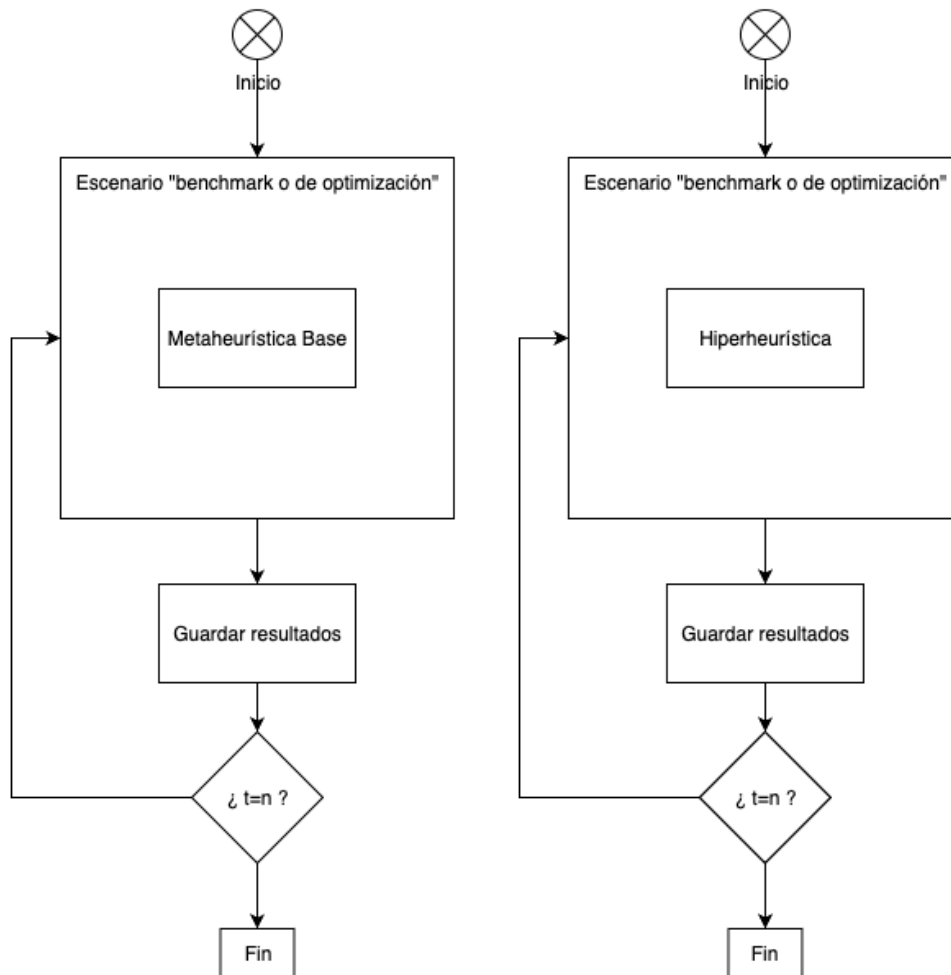


Figura 5.1: Diseño experimental a alto nivel para la evaluación del algoritmo. Cada instancia de problema (Benchmark/Problema de Optimización) se ejecuta N veces para análisis estadístico.

5.1.2. Definición de Variables y Base para la Experimentación

La base de la experimentación se fundamenta en la variación sistemática de los escenarios de prueba y los parámetros clave del algoritmo, con el fin de evaluar la robustez y el rendimiento del controlador de explicabilidad (XAI) propuesto. El diseño experimental se orienta a simular tanto condiciones de evaluación estandarizadas (CEC) como escenarios aplicados de alta complejidad (última milla).

Para cada escenario de optimización, se definen las siguientes variables que controlan la dificultad del problema y el comportamiento del algoritmo:

- Variables de Escenario (Problema): Estas variables definen la dificultad del paisaje de optimización.
 - Dimensión (D): Número de variables de decisión, fundamental para las funciones *benchmark* (e.g., $D = 10, 30, 50$).
 - Carga del Problema (Ruteo): Variables que determinan la complejidad del problema de última milla (e.g., número de clientes, capacidad de los vehículos), ajustadas para generar escenarios de alta carga computacional.
- Variables de Ejecución (Algoritmo): Parámetros que gobiernan la metaheurística y su proceso de búsqueda.
 - Población (P): Tamaño del conjunto de soluciones candidatas o individuos.
 - Criterio de Terminación: Número máximo de Evaluaciones de la Función Objetivo (FES), asegurando que todos los algoritmos compitan bajo el mismo esfuerzo computacional.

5.2. Casos de Estudio y Benchmarks

Esta sección detalla los problemas específicos y los conjuntos de prueba que se utilizarán para validar el algoritmo propuesto, asegurando su comparabilidad con la literatura actual y su capacidad de aplicación en un contexto real.

5.2.1. Funciones de Benchmark Estándar (CEC)

Se utilizará el conjunto de funciones estandarizadas de la CEC), las cuales son la herramienta fundamental en la literatura para la evaluación de algoritmos de optimización global. Estas funciones están diseñadas para presentar una amplia gama de dificultades (por ejemplo, unimodalidad, multimodalidad, composiciones e hibridaciones).

- Propósito: Evaluar la capacidad de exploración y explotación del algoritmo propuesto, en particular, cómo el controlador de explicabilidad ayuda a escapar de óptimos locales en paisajes multimodales complejos.
- Configuración: Las pruebas se realizarán en distintas dimensiones (D) (e.g., 10, 30 y 50), y se definirán los rangos de inicialización según las especificaciones del benchmark para mantener la comparabilidad.

5.2.2. Caso de Estudio Aplicado (Optimización de Última Milla)

Se abordará un caso de estudio representativo de un problema real de optimización aplicado a la logística, específicamente el problema de ruteo de vehículos con capacidad, que modela la optimización de la última milla.

- Propósito: Validar la utilidad práctica del controlador XAI en un problema aplicado de alta complejidad combinatoria y con restricciones discretas de capacidad.
- Descripción Detallada: Se utilizarán instancias clásicas del CVRP (e.g., conjuntos de instancias A, B o P) o instancias sintéticas generadas para simular un escenario de alta carga. El objetivo es minimizar la distancia total recorrida o el costo operativo. La variación en el número de nodos (N) y la capacidad de los vehículos permitirá simular el escenario de alta carga.

5.2.3. Protocolo de Validación e Hipótesis

La primera etapa de la experimentación se centrará en la validación del desempeño del algoritmo, contemplando rigurosamente el protocolo de pruebas planteado en el punto 4.2. El objetivo primordial de esta fase es **rechazar** la hipótesis nula (H_0), para poder **validar** la hipótesis alternativa (H_a), demostrando que la integración del controlador de explicabilidad mejora significativamente el rendimiento.

Dado el carácter estocástico de los algoritmos bioinspirados, los resultados de las ejecuciones suelen violar el supuesto de una distribución normal. Por ello, el análisis estadístico se someterá a un protocolo riguroso de dos pasos para garantizar la validez de las conclusiones:

1. Verificación de Normalidad: Previo a la comparación, se utilizarán las pruebas de Shapiro-Wilk (preferida para muestras pequeñas y moderadas) y Lilliefors (modificación del test de Kolmogórov-Smirnov para parámetros de distribución desconocidos) para comprobar formalmente el supuesto de una distribución Gaussiana en los resultados obtenidos.
2. Contraste de Hipótesis: La alta probabilidad de encontrar no-normalidad en los datos (demostrada por los tests anteriores) justificará plenamente la elección de una herramienta de contraste no paramétrica.

Para la validación de la hipótesis principal, se utilizará un **análisis de comparación de pares** mediante el **Test no paramétrico de Wilcoxon**. Este test es el estándar en la literatura de optimización para determinar si existe una diferencia estadísticamente significativa en el rendimiento (media y mediana) entre dos algoritmos.

Formulación de Hipótesis

Sean μ_{prop} y μ_{ref} los valores promedio de la función objetivo obtenidos por el algoritmo propuesto (con XAI) y el algoritmo de referencia (sin XAI), respectivamente. Asumiendo un problema de minimización, la formulación estadística es:

- **Hipótesis Nula (H_0):** El algoritmo propuesto **no mejora** el rendimiento; es decir, el valor medio de la función objetivo obtenido es igual o superior al valor medio obtenido por el algoritmo de referencia.

$$H_0 : \mu_{\text{prop}} \geq \mu_{\text{ref}}$$

- **Hipótesis Alternativa (H_a):** El algoritmo propuesto **es significativamente mejor**; es decir, el valor medio de la función objetivo obtenido por el algoritmo propuesto es significativamente menor al valor medio obtenido por el algoritmo de referencia.

$$H_a : \mu_{\text{prop}} < \mu_{\text{ref}}$$

5.3. Interpretación de Resultados

La interpretación de los resultados de la etapa experimental se enfocará en dos aspectos clave: la visualización del rendimiento empírico y el contraste estadístico, formal de las hipótesis. Esto permitirá no solo mostrar las diferencias en la calidad de la solución, sino también la robustez y el comportamiento del controlador XAI.

5.3.1. Técnicas de Visualización de Rendimiento

Se emplearán técnicas de visualización estándar en la literatura de optimización, enfocadas en representar la tendencia central, la dispersión y la evolución de la búsqueda:

- **Gráficos de Convergencia (Evolution Graphs):** Muestran el mejor valor de la función objetivo encontrado a lo largo de las Evaluaciones de la Función Objetivo (FES).
 - **Propósito:** Ilustrar la velocidad de convergencia de los algoritmos y, crucialmente, si el algoritmo propuesto (con XAI) logra evitar el estancamiento prematuro en comparación con la versión de referencia.
- **Diagramas de Caja y Bigotes:** Representan la distribución completa de los N resultados finales de las ejecuciones para cada instancia de problema.
 - **Propósito:** Visualizar métricas clave como la mediana, la varianza intercuartílica (robustez) y la presencia de valores atípicos. Son esenciales para confirmar visualmente la mejora en la robustez antes de la prueba de Wilcoxon.

- Gráfico de contribución de LIme
- Visualización del Controlador XAI: Se utilizarán gráficos específicos (e.g., histogramas o gráficos de líneas de tiempo) para mostrar:
 - La frecuencia con la que el controlador de explicabilidad activa sus mecanismos de escape.
 - La justificación de las decisiones del controlador (e.g., qué reglas se activaron antes de una mejora significativa).

5.3.2. Interpretación y Análisis Estadístico

La interpretación de los resultados se fundamentará en el protocolo estadístico definido previamente:

1. Análisis Descriptivo: Se utilizarán las métricas de media, mediana y desviación estándar para proporcionar una evaluación empírica inicial de la calidad de la solución (media/mediana) y la fiabilidad del algoritmo.
2. Inferencia Estadística (Test de Wilcoxon): La interpretación clave se centrará en el **p-valor** resultante de la prueba de Wilcoxon, a un nivel de significancia de $\alpha = 0.05$.
 - **Si $p < 0.05$:** Se **rechaza** la hipótesis nula (H_0). Esto significa que existe evidencia estadísticamente significativa en contra de H_0 y, por lo tanto, se asume la hipótesis alternativa (H_a), concluyendo que la mejora del rendimiento del algoritmo propuesto es **estadísticamente significativa** (el objetivo de la tesis).
 - **Si $p \geq 0.05$:** Se **NO rechaza** la hipótesis nula (H_0). Se concluye que la evidencia en los datos no es lo bastante fuerte como para descartar H_0 , y la diferencia observada en el rendimiento no es estadísticamente significativa.
3. Análisis de Parámetros: Se interpretarán las comparaciones por pares de Wilcoxon para justificar la elección de la configuración óptima, asegurando que los parámetros seleccionados no solo rinden bien, sino que son significativamente mejores que las alternativas.

Bibliografía

- [1] H. Rezk, A. Ghani Olabi, T. Wilberforce, and E. Taha Sayed, “Metaheuristic optimization algorithms for real-world electrical and civil engineering application: A review,” *Results in Engineering*, vol. 23, p. 102437, Sep. 2024. [Online]. Available: <http://dx.doi.org/10.1016/j.rineng.2024.102437>
- [2] Y. Qawqzeh, M. T. Alharbi, A. Jaradat, and K. N. Abdul Sattar, “A review of swarm intelligence algorithms deployment for scheduling and optimization in cloud computing environments,” *PeerJ Computer Science*, vol. 7, p. e696, Aug. 2021. [Online]. Available: <http://dx.doi.org/10.7717/peerj-cs.696>
- [3] S. Zhao, T. Zhang, S. Ma, and M. Wang, “Sea-horse optimizer: a novel nature-inspired meta-heuristic for global optimization problems,” *Applied Intelligence*, vol. 53, no. 10, p. 11833–11860, Sep. 2022. [Online]. Available: <http://dx.doi.org/10.1007/s10489-022-03994-3>
- [4] J. Almeida, J. Soares, F. Lezama, S. Limmer, T. Rodemann, and Z. Vale, “A systematic review of explainability in computational intelligence for optimization,” *Computer Science Review*, vol. 57, p. 100764, Aug. 2025. [Online]. Available: <http://dx.doi.org/10.1016/j.cosrev.2025.100764>
- [5] M. Hjeij and A. Vilks, “A brief history of heuristics: how did research on heuristics evolve?” *Humanities and Social Sciences Communications*, vol. 10, no. 1, Feb. 2023. [Online]. Available: <http://dx.doi.org/10.1057/s41599-023-01542-z>
- [6] J. Kennedy and R. Eberhart, “Particle swarm optimization,” in *Proceedings of ICNN’95 - International Conference on Neural Networks*, ser. ICNN-95, vol. 4. IEEE, 1995, p. 1942–1948. [Online]. Available: <http://dx.doi.org/10.1109/ICNN.1995.488968>
- [7] I. Tariq, H. A. AlSattar, A. A. Zaidan, B. B. Zaidan, M. R. Abu Bakar, R. T. Mohammed, O. S. Albahri, M. A. Alsalem, and A. S. Albahri, “Mogsabat: a metaheuristic hybrid algorithm for solving multi-objective optimisation problems,”

- Neural Computing and Applications*, vol. 32, no. 8, p. 3101–3115, Oct. 2018. [Online]. Available: <http://dx.doi.org/10.1007/s00521-018-3808-3>
- [8] M. Črepinšek, M. Mernik, M. Beković, M. Pintarič, M. Moravec, and M. Ravber, “Overcoming stagnation in metaheuristic algorithms with msma’s adaptive meta-level partitioning,” *Mathematics*, vol. 13, no. 11, p. 1803, May 2025. [Online]. Available: <http://dx.doi.org/10.3390/math13111803>
- [9] I. Oueslati, M. Hammami, I. Nouaouri, A. Azzouz, L. B. Said, and H. Allaoui, *A Honey Bee Mating Optimization HyperHeuristic for Patient Admission Scheduling Problem*. Springer Nature Switzerland, 2024, p. 89–104. [Online]. Available: http://dx.doi.org/10.1007/978-3-031-69257-4_7
- [10] F. Zhao, S. Di, and L. Wang, “A hyperheuristic with q-learning for the multiobjective energy-efficient distributed blocking flow shop scheduling problem,” *IEEE Transactions on Cybernetics*, vol. 53, no. 5, p. 3337–3350, May 2023. [Online]. Available: <http://dx.doi.org/10.1109/TCYB.2022.3192112>
- [11] N. Zhu, F. Zhao, and J. Cao, “A hyperheuristic and reinforcement learning guided meta-heuristic algorithm recommendation,” in *2024 27th International Conference on Computer Supported Cooperative Work in Design (CSCWD)*. IEEE, May 2024, p. 1061–1066. [Online]. Available: <http://dx.doi.org/10.1109/CSCWD61410.2024.10580058>
- [12] L. Longo, M. Brcic, F. Cabitza, J. Choi, R. Confalonieri, J. D. Ser, R. Guidotti, Y. Hayashi, F. Herrera, A. Holzinger, R. Jiang, H. Khosravi, F. Lecue, G. Malgieri, A. Páez, W. Samek, J. Schneider, T. Speith, and S. Stumpf, “Explainable artificial intelligence (xai) 2.0: A manifesto of open challenges and interdisciplinary research directions,” *Information Fusion*, vol. 106, p. 102301, Jun. 2024. [Online]. Available: <http://dx.doi.org/10.1016/j.inffus.2024.102301>
- [13] K. Kalasampath, K. N. Spoorthi, S. Sajeev, S. S. Kuppa, K. Ajay, and A. Maruthamuthu, “A literature review on applications of explainable artificial intelligence (xai),” *IEEE Access*, vol. 13, p. 41111–41140, 2025. [Online]. Available: <http://dx.doi.org/10.1109/ACCESS.2025.3546681>
- [14] P. Cao and J. Tang, “A reinforcement learning hyper-heuristic in multi-objective single point search with application to structural fault identification,” 2018. [Online]. Available: <https://arxiv.org/abs/1812.07958>
- [15] D. Gusfield, *Integer Linear Programming in Computational and Systems Biology: An Entry-Level Text and Course*. Cambridge University Press, May 2019. [Online]. Available: <http://dx.doi.org/10.1017/9781108377737>

- [16] E.-Y. Youness and L. Lahoussine, “Necessary optimality conditions for a semivectorial bilevel optimization problem via a revisited charnes–cooper scalarization,” *Optimization*, vol. 74, no. 1, p. 165–188, Aug. 2023. [Online]. Available: <http://dx.doi.org/10.1080/02331934.2023.2249916>
- [17] O. L. Londoño Palacio, L. F. Maldonado Granados, and L. C. Calderón Villafañez, “Guías para construir estados del arte,” 2014.
- [18] A. W. Mohamed, K. M. Sallam, P. Agrawal, A. A. Hadi, and A. K. Mohamed, “Evaluating the performance of meta-heuristic algorithms on cec 2021 benchmark problems,” *Neural Computing and Applications*, vol. 35, no. 2, p. 1493–1517, Sep. 2022. [Online]. Available: <http://dx.doi.org/10.1007/s00521-022-07788-z>
- [19] S. S. Shapiro and M. B. Wilk, “An analysis of variance test for normality (complete samples),” *Biometrika*, vol. 52, no. 3-4, pp. 591–611, 1965.
- [20] H. W. Lilliefors, “On the kolmogorov-smirnov test for normality with mean and variance unknown,” *Journal of the American Statistical Association*, vol. 62, no. 318, p. 399–402, Jun. 1967. [Online]. Available: <http://dx.doi.org/10.1080/01621459.1967.10482916>
- [21] J. Derrac, S. García, D. Molina, and F. Herrera, “A practical tutorial on the use of nonparametric statistical tests as a methodology for comparing evolutionary and swarm intelligence algorithms,” *Swarm and Evolutionary Computation*, vol. 1, no. 1, p. 3–18, Mar. 2011. [Online]. Available: <http://dx.doi.org/10.1016/j.swevo.2011.02.002>
- [22] J. Waldrop and J. J. Dunlap, “Ce: Beyond pico—a new question simplifies the search for evidence,” *AJN, American Journal of Nursing*, vol. 124, no. 3, p. 34–37, Feb. 2024. [Online]. Available: <http://dx.doi.org/10.1097/01.NAJ.0001007676.91191.dd>
- [23] W. Zhang, X. Bao, X. Hao, and M. Gen, “Metaheuristics for multi-objective scheduling problems in industry 4.0 and 5.0: a state-of-the-arts survey,” *Frontiers in Industrial Engineering*, vol. 3, Jan. 2025. [Online]. Available: <http://dx.doi.org/10.3389/fieng.2025.1540022>
- [24] T. Hamadneh, “Application of orangutan optimization algorithm for feature selection problems,” *INASS Express*, vol. 1, no. 1, p. 1–9, Apr. 2025. [Online]. Available: <http://dx.doi.org/10.22266/inassexpress.2025.001>
- [25] J. Barrera-García, F. Cisternas-Caneo, B. Crawford, M. Gómez Sánchez, and R. Soto, “Feature selection problem and metaheuristics: A systematic literature review about its formulation, evaluation and applications,” *Biomimetics*, vol. 9, no. 1, p. 9, Dec. 2023. [Online]. Available: <http://dx.doi.org/10.3390/biomimetics9010009>

- [26] E. V. Altay, O. Altay, and Y. Özçevik, “A comparative study of metaheuristic optimization algorithms for solving real-world engineering design problems,” *Computer Modeling in Engineering & Sciences*, vol. 139, no. 1, p. 1039–1094, 2024. [Online]. Available: <http://dx.doi.org/10.32604/cmescs.2023.029404>
- [27] X. Liu, “Mathematical scheduling model of complex industrial process combining swarm intelligence algorithm and swarm dimension reduction technology,” *Results in Engineering*, vol. 21, p. 101796, Mar. 2024. [Online]. Available: <http://dx.doi.org/10.1016/j.rineng.2024.101796>
- [28] K. Rajwar, K. Deep, and S. Das, “An exhaustive review of the metaheuristic algorithms for search and optimization: taxonomy, applications, and open challenges,” *Artificial Intelligence Review*, vol. 56, no. 11, p. 13187–13257, Apr. 2023. [Online]. Available: <http://dx.doi.org/10.1007/s10462-023-10470-y>
- [29] R. Kamal, E. Amin, D. S. AbdElminaam, and R. Ismail, “A comprehensive survey on meta-heuristic algorithms for feature selection in high-dimensional data: Challenges, applications, and future directions,” in *2024 International Mobile, Intelligent, and Ubiquitous Computing Conference (MIUCC)*. IEEE, Nov. 2024, p. 84–90. [Online]. Available: <http://dx.doi.org/10.1109/MIUCC62295.2024.10783538>
- [30] E. Osaba, E. Villar-Rodriguez, J. Del Ser, A. J. Nebro, D. Molina, A. LaTorre, P. N. Suganthan, C. A. Coello Coello, and F. Herrera, “A tutorial on the design, experimentation and application of metaheuristic algorithms to real-world optimization problems,” *Swarm and Evolutionary Computation*, vol. 64, p. 100888, Jul. 2021. [Online]. Available: <http://dx.doi.org/10.1016/j.swevo.2021.100888>
- [31] F. A. Hashim, R. R. Mostafa, R. A. Khurma, R. Qaddoura, and P. A. Castillo, “A new approach for solving global optimization and engineering problems based on modified sea horse optimizer,” *Journal of Computational Design and Engineering*, vol. 11, no. 1, p. 73–98, Dec. 2023. [Online]. Available: <http://dx.doi.org/10.1093/jcde/qwae001>
- [32] B. Gülmez, “A hybrid approach for stock market price forecasting using long short-term memory and seahorse optimization algorithm,” *Annals of Data Science*, May 2025. [Online]. Available: <http://dx.doi.org/10.1007/s40745-025-00609-9>
- [33] Z. Li, S. Qu, Y. Xu, X. Hao, and N. Lin, “Enhanced sea horse optimization algorithm for hyperparameter optimization of agricultural image recognition,” *Mathematics*, vol. 12, no. 3, p. 368, Jan. 2024. [Online]. Available: <http://dx.doi.org/10.3390/math12030368>

- [34] E. H. Houssein, M. R. Saad, E. undefinedelik, G. Hu, A. A. Ali, and H. Shaban, “An enhanced sea-horse optimizer for solving global problems and cluster head selection in wireless sensor networks,” *Cluster Computing*, vol. 27, no. 6, p. 7775–7802, Apr. 2024. [Online]. Available: <http://dx.doi.org/10.1007/s10586-024-04368-9>
- [35] A. Erduman, G. Hazar, and E. B. Aydın, “A sea horse optimization-based approach for pem fuel cell model parameter estimation,” *Applied Sciences*, vol. 15, no. 15, p. 8316, Jul. 2025. [Online]. Available: <http://dx.doi.org/10.3390/app15158316>
- [36] H. M. Hasanien, I. Alsaleh, M. Tostado-Véliz, M. Zhang, A. Alateeq, F. Jurado, and A. Alassaf, “Hybrid particle swarm and sea horse optimization algorithm-based optimal reactive power dispatch of power systems comprising electric vehicles,” *Energy*, vol. 286, p. 129583, Jan. 2024. [Online]. Available: <http://dx.doi.org/10.1016/j.energy.2023.129583>
- [37] Y.-C. Wang, H.-M. Song, J.-S. Wang, Y.-W. Song, Y.-L. Qi, and X.-R. Ma, “Gog-mbsho: multi-strategy fusion binary sea-horse optimizer with gaussian transfer function for feature selection of cancer gene expression data,” *Artificial Intelligence Review*, vol. 57, no. 12, Oct. 2024. [Online]. Available: <http://dx.doi.org/10.1007/s10462-024-10954-5>
- [38] C. Andic, S. Ozumcan, M. Varan, and A. Ozturk, “A novel sea horse optimizer based load frequency controller for two-area power system with pv and thermal units,” *International Journal of Robotics and Control Systems*, vol. 4, no. 2, p. 606–627, May 2024. [Online]. Available: <http://dx.doi.org/10.31763/ijrcs.v4i2.1341>
- [39] A. G. Hussien, R. A. Khurma, A. Alzaqebah, M. Amin, and F. A. Hashim, “Novel memetic of beluga whale optimization with self-adaptive exploration–exploitation balance for global optimization and engineering problems,” *Soft Computing*, vol. 27, no. 19, p. 13951–13989, Jun. 2023. [Online]. Available: <http://dx.doi.org/10.1007/s00500-023-08468-3>
- [40] F. Lagos and J. Pereira, “Multi-armed bandit-based hyper-heuristics for combinatorial optimization problems,” *European Journal of Operational Research*, vol. 312, no. 1, p. 70–91, Jan. 2024. [Online]. Available: <http://dx.doi.org/10.1016/j.ejor.2023.06.016>
- [41] A. Seyyedabbasi, W. Z. Tareq Tareq, and N. Bacanin, “An effective hybrid metaheuristic algorithm for solving global optimization algorithms,” *Multimedia Tools and Applications*, vol. 83, no. 37, p. 85103–85138, May 2024. [Online]. Available: <http://dx.doi.org/10.1007/s11042-024-19437-9>
- [42] S. Lin and H. Xu, “An adaptive differential evolution algorithm based on adaptive evolution strategy and diversity enhancement,” *Results in Engineering*, vol. 27,

- p. 106551, Sep. 2025. [Online]. Available: <http://dx.doi.org/10.1016/j.rineng.2025.106551>
- [43] T. Dokeroglu, T. Kucukyilmaz, and E.-G. Talbi, "Hyper-heuristics: A survey and taxonomy," *Computers & Industrial Engineering*, vol. 187, p. 109815, Jan. 2024. [Online]. Available: <http://dx.doi.org/10.1016/j.cie.2023.109815>
- [44] B. H. Abed-Alguni, "Evomapx: An explainable framework for metaheuristic optimization algorithms," *Expert Systems with Applications*, vol. 298, p. 129514, Mar. 2026. [Online]. Available: <http://dx.doi.org/10.1016/j.eswa.2025.129514>
- [45] L. Sun, Z. Song, J. Liu, H. Ji, G. Guo, N. Matsveichuk, and Y. Sotskov, "Hierarchically controlled differential evolution algorithm," *Expert Systems with Applications*, vol. 290, p. 128383, Sep. 2025. [Online]. Available: <http://dx.doi.org/10.1016/j.eswa.2025.128383>
- [46] E. Rodríguez-Esparza, A. D. Masegosa, D. Oliva, and E. Onieva, "A new hyper-heuristic based on adaptive simulated annealing and reinforcement learning for the capacitated electric vehicle routing problem," *Expert Systems with Applications*, vol. 252, p. 124197, Oct. 2024. [Online]. Available: <http://dx.doi.org/10.1016/j.eswa.2024.124197>
- [47] Y. Zhang, G. Chen, L. Cheng, Q. Wang, and Q. Li, "Methods to balance the exploration and exploitation in differential evolution from different scales: A survey," *Neurocomputing*, vol. 561, p. 126899, Dec. 2023. [Online]. Available: <http://dx.doi.org/10.1016/j.neucom.2023.126899>
- [48] T. Barros-Everett, E. Montero, and N. Rojas-Morales, "Parameter prediction for metaheuristic algorithms solving routing problem instances using machine learning," *Applied Sciences*, vol. 15, no. 6, p. 2946, Mar. 2025. [Online]. Available: <http://dx.doi.org/10.3390/app15062946>
- [49] D. Hu, Y. Yang, H. Dai, C. Tang, and J. Xie, "An interpretable machine learning method for fault diagnosis of oil-immersed transformers based on edge inference," *International Journal of Electrical & Energy Systems*, vol. 168, p. 110647, Jul. 2025. [Online]. Available: <http://dx.doi.org/10.1016/j.ijepes.2025.110647>
- [50] Z. Wang, X. Chen, Y. Wu, L. Jiang, S. Lin, and G. Qiu, "A robust and interpretable ensemble machine learning model for predicting healthcare insurance fraud," *Scientific Reports*, vol. 15, no. 1, Jan. 2025. [Online]. Available: <http://dx.doi.org/10.1038/s41598-024-82062-x>

- [51] A. Bolufé-Röhler and D. Tamayo-Vera, “Machine learning for enhancing metaheuristics in global optimization: A comprehensive review,” *Mathematics*, vol. 13, no. 18, p. 2909, Sep. 2025. [Online]. Available: <http://dx.doi.org/10.3390/math13182909>
- [52] H. Nakagawa, S. Sumita, R. Iida, and T. Tsuchiya, “An xai-based meta-parameter tuning for time-series forecasting,” *Annals of Mathematics and Artificial Intelligence*, Mar. 2025. [Online]. Available: <http://dx.doi.org/10.1007/s10472-025-09973-x>
- [53] M. Medani, G. M. S. Elhessewi, M. Alqahtani, S. A. Asklany, S. Alamro, D. Albalawneh, M. Alshammeri, and M. Assiri, “Leveraging explainable artificial intelligence with ensemble of deep learning model for dementia prediction to enhance clinical decision support systems,” *Scientific Reports*, vol. 15, no. 1, May 2025. [Online]. Available: <http://dx.doi.org/10.1038/s41598-025-97102-3>
- [54] M. Almsallti, A. B. Alzubi, and O. R. Adegboye, “Hybrid metaheuristic optimized extreme learning machine for sustainability focused co2 emission prediction using globalization-driven indicators,” *Sustainability*, vol. 17, no. 15, p. 6783, Jul. 2025. [Online]. Available: <http://dx.doi.org/10.3390/su17156783>
- [55] S. Salamon and W. Ksiazek, “Capsule neural networks with bayesian optimization for pediatric pneumonia detection from chest x-ray images,” *Journal of Clinical Medicine*, vol. 14, no. 20, p. 7212, Oct. 2025. [Online]. Available: <http://dx.doi.org/10.3390/jcm14207212>
- [56] R. Hernández Sampieri, C. Fernández Collado, and P. Baptista Lucio, *Metodología de la investigación: Las rutas cuantitativa, cualitativa y mixta*, sexta ed. McGraw-Hill Interamericana, 2018.
- [57] H. Bakır, “Comparative performance analysis of metaheuristic search algorithms in parameter extraction for various solar cell models,” *Environmental Challenges*, vol. 11, p. 100720, Apr. 2023. [Online]. Available: <http://dx.doi.org/10.1016/j.envc.2023.100720>